

AMAZON VPC FLOW LOGS ANALYSIS FOR TRAFFIC AUDITING

AWS-Based Network Traffic Monitoring and Audit Analysis

PROJECT DOCUMENTATION

Final Documentation Prefix

Project	Amazon VPC Flow Logs Analysis for Traffic Auditing
Team	Team-382
Core approach	VPC Flow Logs → S3 → Lambda → DynamoDB → API Gateway → Dashboard
Focus	Network visibility and traffic auditing

Contents

1. Executive Summary
2. Problem Statement
3. Project Objectives
4. Solution Understanding
5. Architecture and Design
6. AWS Services and Their Roles
7. End-to-End Workflow
8. Phase Overview
9. Cost-First Design
10. Security and IAM Approach
11. Dashboard and Audit Capabilities
12. Verification and Expected Outcome
13. Future Scope
14. Project Walkthrough

Note: The detailed implementation screenshots and step-by-step AWS console evidence are intended to follow these prefix pages. The walkthrough pages can be appended after this section without changing the prefix.

1. Executive Summary

Amazon VPC Flow Logs Analysis for Traffic Auditing is an AWS-based network monitoring and audit solution designed to provide systematic visibility into traffic flowing through a VPC. The project captures VPC Flow Logs, stores the raw records in Amazon S3, processes the delivered log objects with AWS Lambda, and stores structured traffic records in Amazon DynamoDB for efficient querying.

The solution also incorporates Amazon SNS for threshold-based email alerting when a newly created S3 flow-log object exceeds the configured size threshold. A query layer using Lambda and API Gateway exposes the structured data to a lightweight dashboard where traffic can be filtered by action, source IP, and destination port.

Project focus: visibility and auditing of network traffic rather than automatic threat detection.

Solution in one line

Raw VPC network activity is converted into structured, queryable traffic-audit information through an event-driven AWS pipeline.

2. Problem Statement

- Network activity inside a VPC can be difficult to observe systematically without centralized traffic records.
- Large volumes of flow-log information make manual inspection inefficient.
- Important auditing fields such as source, destination, protocol, ports, action, packets, bytes, and timestamps need to be available in a structured form.
- Audit investigations benefit from searchable and filterable traffic records rather than raw log files alone.

3. Project Objectives

- Enable and collect Amazon VPC Flow Logs.
- Centralize raw traffic records in Amazon S3.
- Automatically process and transform the delivered .log.gz records.
- Store structured flow information in Amazon DynamoDB.
- Support filtering and querying using DynamoDB indexes.
- Provide a lightweight dashboard for traffic-audit analysis.
- Use IAM least-privilege permissions for service-to-service communication.
- Keep the implementation cost-conscious and avoid unnecessary paid or complex services.

4. Solution Understanding

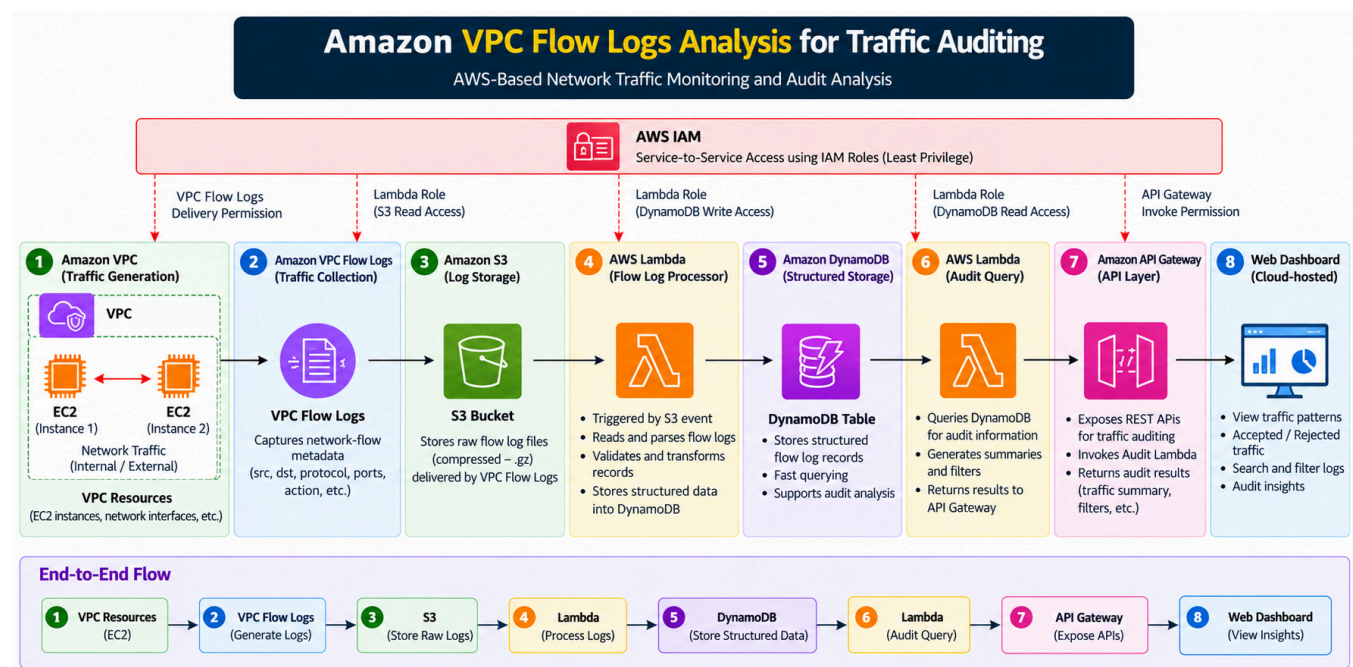
The project is organized as a sequence of cloud services, where each service has a focused responsibility. Network traffic is first generated in the test VPC environment. VPC Flow Logs capture the traffic metadata and deliver compressed log objects to the S3 bucket. The S3 ObjectCreated event then starts the processing of Lambda.

The processing Lambda checks the object size, optionally publishes an SNS notification for an oversized object, decompresses the log, parses the flow-record fields, validates and transforms the data, and writes the resulting records to DynamoDB. A separate query path retrieves records through DynamoDB indexes and exposes them through API Gateway.

5. Architecture and Design

Layer	Flow
Collection	EC2 / VPC → VPC Flow Logs → S3
Processing	S3 ObjectCreated → FlowLogProcessor Lambda → DynamoDB
Alerting	Oversized S3 object → Lambda → SNS → Email
Query	Dashboard → API Gateway → Query Lambda → DynamoDB
Indexes	ActionIndex, SourceIpIndex, DestinationPortIndex
Support	IAM, CloudWatch, CloudShell, GitHub

EC2/Test Traffic → VPC Flow Logs → S3 → Lambda Processing → DynamoDB → Query Lambda → API Gateway → Dashboard



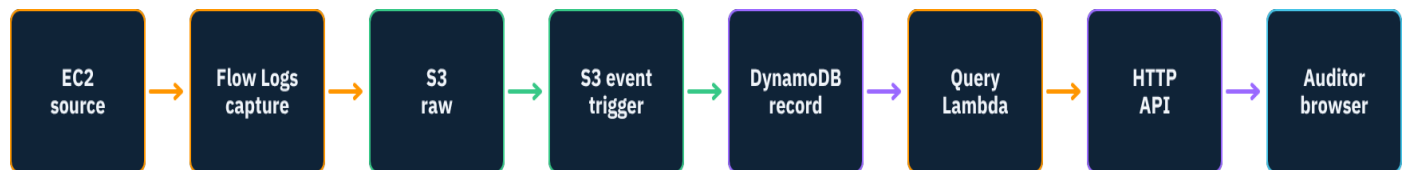
6. AWS Services and Their Roles

Service	Role
Amazon VPC	Provides the network environment in which the test traffic is generated.
Amazon VPC Flow Logs	Captures network-flow metadata, including interface, source/destination, ports, protocol, packets, bytes, timestamps, action, and log status.
Amazon S3	Stores raw VPC flow log objects and serves as the project artifact/storage layer.
AWS Lambda	Processes incoming S3 log objects and provides the query layer for the dashboard.

Amazon DynamoDB	Stores normalized flow records and supports indexed queries.
Amazon API Gateway	Provides the API entry point for dashboard requests.
Amazon SNS	Sends an email alert when the configured S3 object-size threshold is reached.
Amazon CloudWatch	Provides Lambda execution logs and operational verification.
AWS IAM	Controls service-to-service permissions using least-privilege roles and policies.
AWS CloudShell	Provides a cloud-based environment for AWS CLI, GitHub, and deployment operations.
GitHub	Maintains the project source code and deployment-related files.

7. End-to-End Workflow

1. Generate controlled network traffic in the test VPC/EC2 environment.
2. VPC Flow Logs capture the traffic metadata.
3. Flow Log objects are delivered to the project S3 bucket under the AWSLogs path.
4. S3 ObjectCreated triggers the FlowLogProcessor Lambda.
5. Lambda checks the compressed object size.
6. If the threshold is exceeded, Lambda publishes an SNS alert; otherwise, processing continues normally.
7. Lambda decompresses the .log.gz object and parses the flow records.
8. Valid records are transformed and written to DynamoDB.
9. DynamoDB indexes support Action, Source IP, and Destination Port filtering.
10. The dashboard sends requests through API Gateway to the query Lambda.
11. Query results are returned and presented as traffic-audit information.



8. Phase Overview

Phase 1—Foundation: S3 bucket, VPC, subnet, Internet Gateway, route table, security group, EC2 test instance, CloudShell/GitHub foundation.

Phase 2—Flow Logs: Generate traffic, enable VPC Flow Logs, deliver .log.gz records to S3, and verify the raw records.

Phase 3 — Project Core: Implement S3 → Lambda → DynamoDB processing and SNS threshold alerting.

Phase 4—Query/API Layer: Implement DynamoDB indexes and query Lambda and API Gateway.

Phase 5—Dashboard: Build the lightweight browser dashboard for summary cards, filters, and result tables.

Phase 7—Security + Cost Audit: Lock down S3, verify IAM, review API configuration, remove unnecessary exposure, and verify cost-sensitive settings.

9. Cost-First Design

Cost control is treated as a primary design constraint. The implementation avoids services and settings that are not necessary for the demonstration and keeps the core resources at small, controlled configurations.

Area	Cost-conscious approach
DynamoDB	Provisioned capacity; cost-first target of 1 RCU / 1 WCU for the base table, with required GSIs kept at the minimum practical provisioned capacity. Auto Scaling OFF.
Lambda	128 MB memory, 30-second timeout, arm64, Provisioned Concurrency OFF, and no VPC attachment.
SNS	One Standard topic and one email subscription; no SMS or unnecessary delivery integrations.
Networking	Avoid unnecessary NAT Gateway or other paid networking components.
Analytics	Athena is not part of the implementation; DynamoDB is used as the structured query store.
Testing	Generate controlled traffic and avoid unnecessary repeated log generation.

Important implementation note: DynamoDB indexes are created one at a time and allowed to reach ACTIVE before another index is created, avoiding simultaneous index-operation failures.

10. Security and IAM Approach

The project uses least-privilege service permissions rather than broad administrative access. The ingestion Lambda requires only the permissions necessary to read flow-log objects from S3, write records to DynamoDB, publish to SNS, and write execution logs.

- Do not use AdministratorAccess for the Lambda execution role.
- Do not attach broad S3FullAccess, DynamoDBFullAccess or SNSFullAccess policies.
- Restrict S3 access to the required flow-log object path.
- Restrict DynamoDB access to the project table and required operations.
- Restrict SNS permission to publishing to the project alert topic.
- Keep EC2 SSH access restricted to the required source rather than opening SSH to the world.

11. Dashboard and Audit Capabilities

- Summary cards for total returned records, ACCEPT records and REJECT records.
- Action filter for ACCEPT / REJECT.

- Source IP filtering.
- Destination-port filtering.
- Results table containing timestamp, source IP, destination IP, source port, destination port, protocol, packets, bytes and action.
- Security/audit indication for REJECT traffic.
- Visibility of the SNS large-object alert mechanism.

12. Verification and Expected Outcome

The project walkthrough demonstrates the following end-to-end verification points:

- VPC Flow Logs were delivered to S3.
- S3 ObjectCreated events triggered the processing Lambda.
- CloudWatch showed successful real Lambda invocations.
- The documented test invocations processed 81 and 70 records, respectively.
- Structured records were written to DynamoDB.
- The SNS threshold test successfully produced an email notification.
- The dashboard successfully returned filtered audit records.

Expected outcomes

- Improved network visibility.
- Centralized and structured traffic records.
- Faster and more systematic traffic-audit analysis.
- Clear ACCEPT / REJECT visibility.
- A reproducible cloud-native demonstration of service-to-service communication.

13. Future Scope

- Richer dashboard filters and saved queries.
- Longer-term retention and export workflows.
- Cross-account traffic-audit views.
- Trend reporting for network governance.

14. Project Resources

GitHub Repository

The complete source code, AWS configuration files, deployment scripts, and supporting project resources are available in the GitHub repository.

GitHub: [Link](#)

Project Presentation

The project presentation contains the problem statement, proposed solution, system architecture, AWS services, workflow, and expected outcomes.

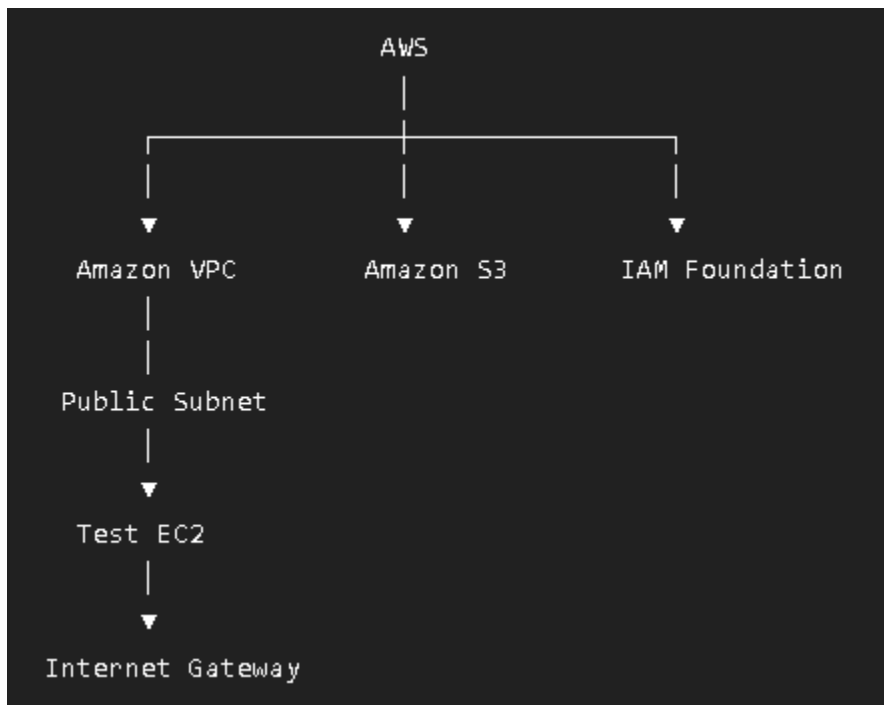
Presentation: [Link](#)

15. Project Walkthrough

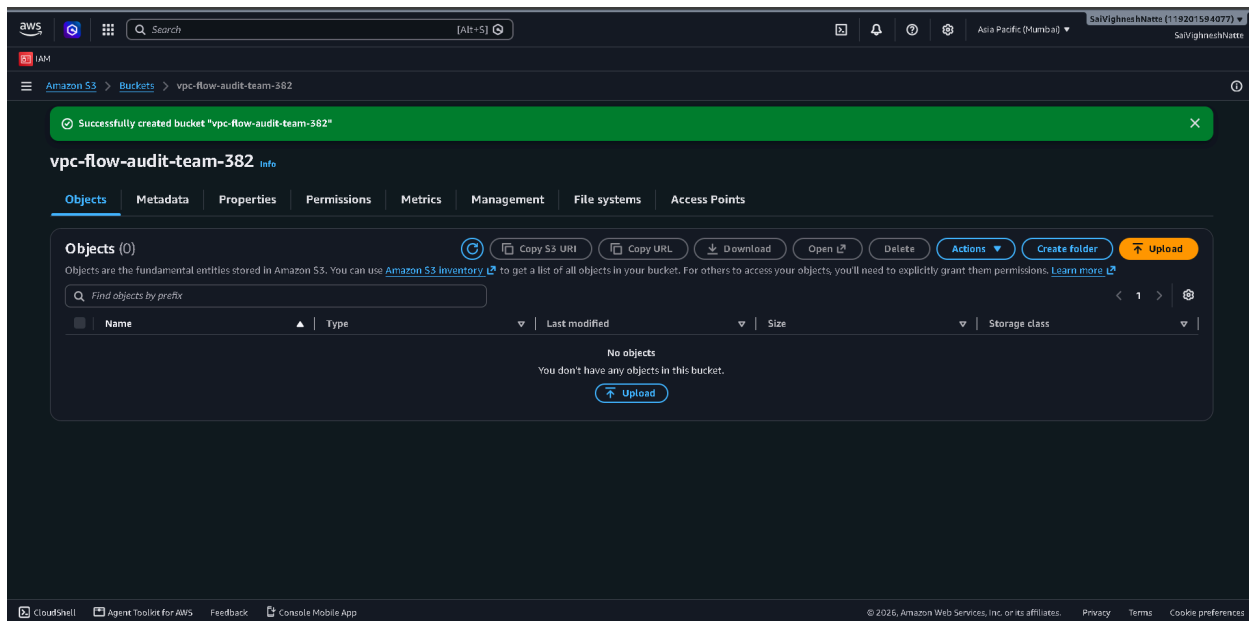
Detailed implementation evidence follows this page. The complete AWS console walkthrough, configuration screenshots, verification screenshots, testing evidence, and dashboard screenshots can be appended here from the supplied project walkthrough document.

Phase 1

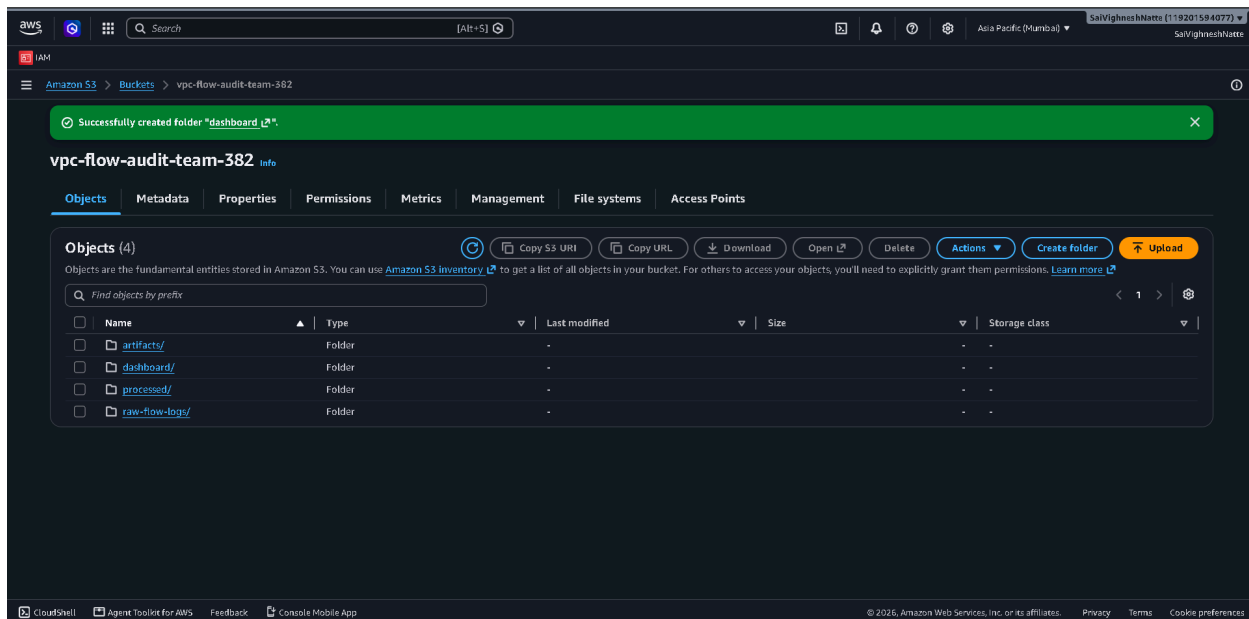
Objective:



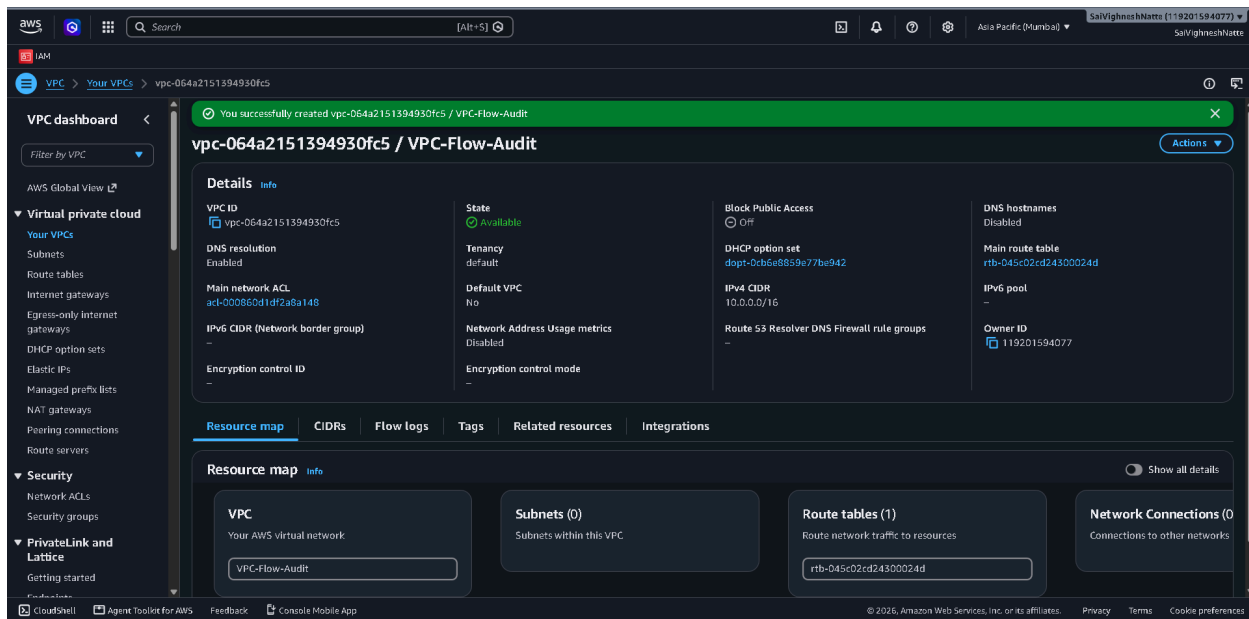
Create the S3 bucket



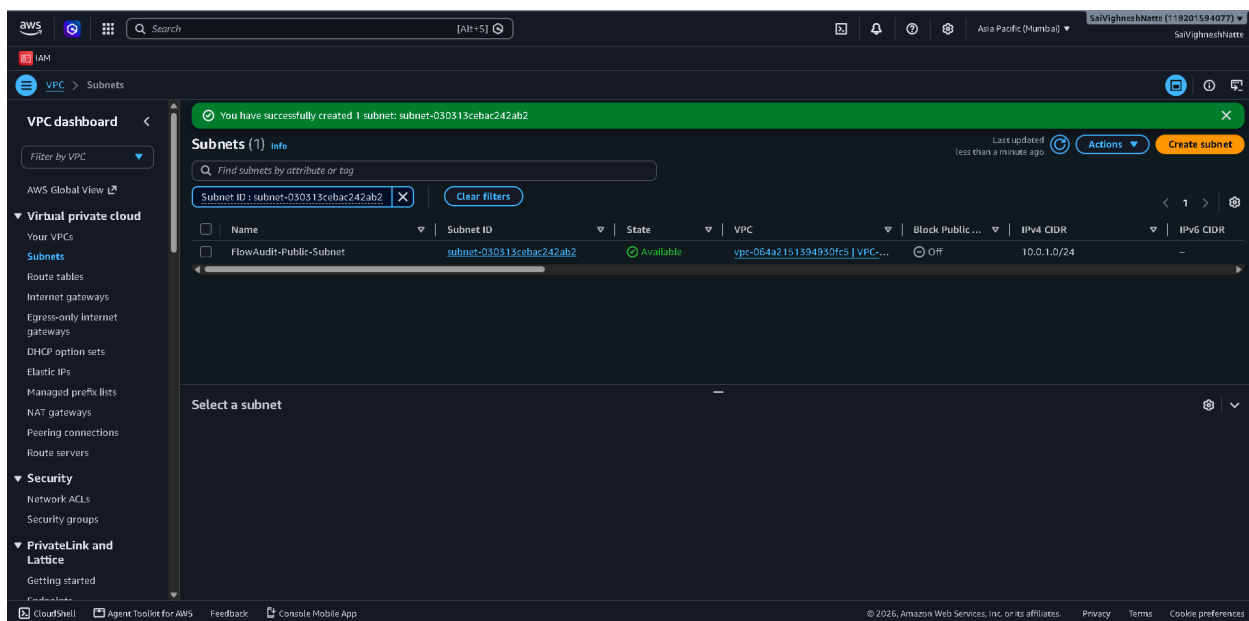
Folder creation



Create VPC (VPC-only)



Subnet creation



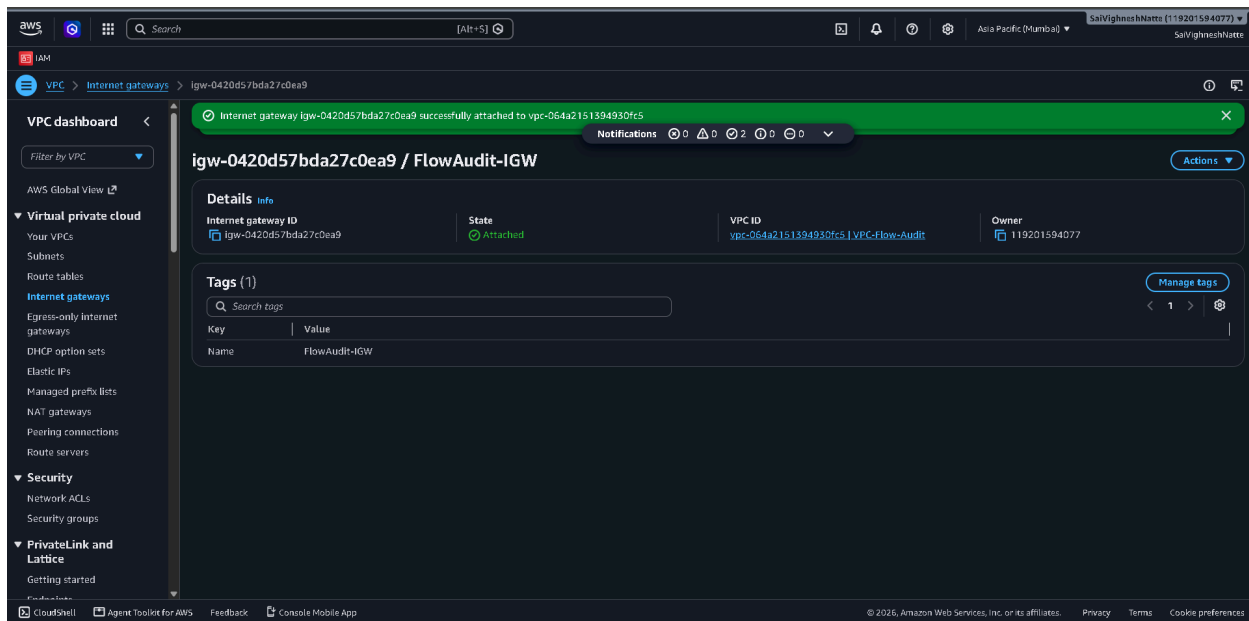
Internet Gateway settings:

Create an IGW.

The screenshot shows the AWS Management Console interface for an Internet Gateway. The left sidebar contains navigation links for VPC, Internet gateways, and other services. The main content area displays the details for the Internet Gateway 'igw-0420d57bda27c0ea9 / FlowAudit-IGW'. The 'Details' section shows the Internet gateway ID, State (Detached), VPC ID (None), and Owner (119201594077). The 'Tags' section shows one tag with the key 'Name' and value 'FlowAudit-IGW'. A green notification bar at the top states: 'The following Internet gateway was created: igw-0420d57bda27c0ea9 - FlowAudit-IGW. You can now attach to a VPC to enable the VPC to communicate with the internet.' An 'Attach to a VPC' button is visible in the top right of the notification bar.

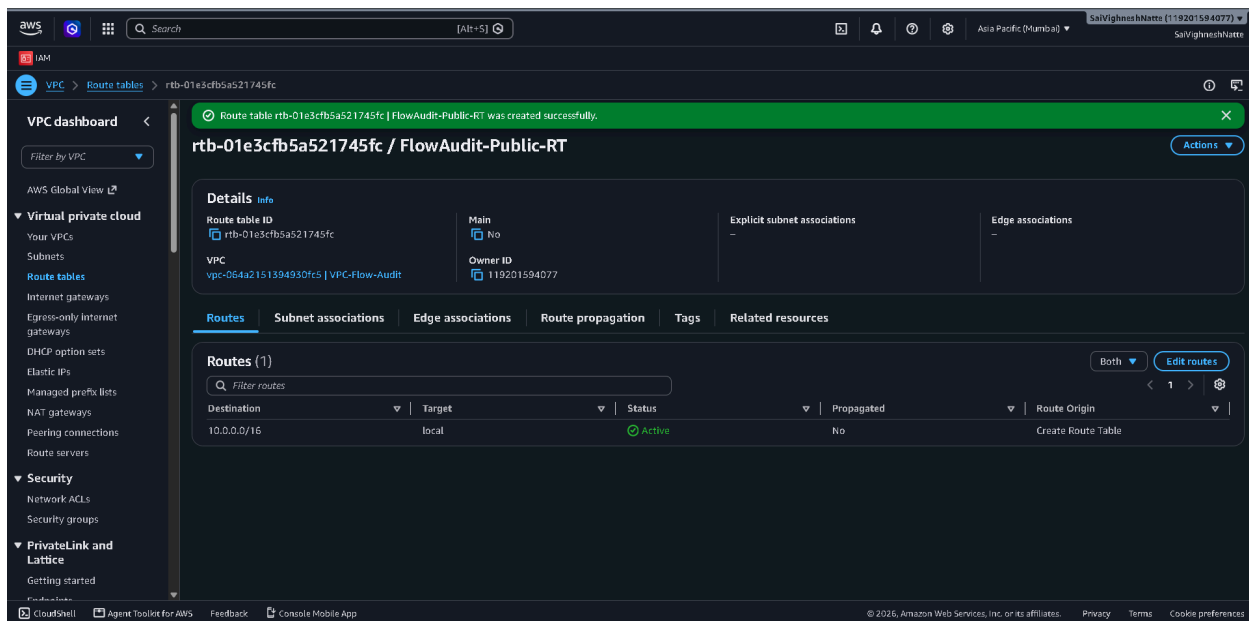
Attach the VPC

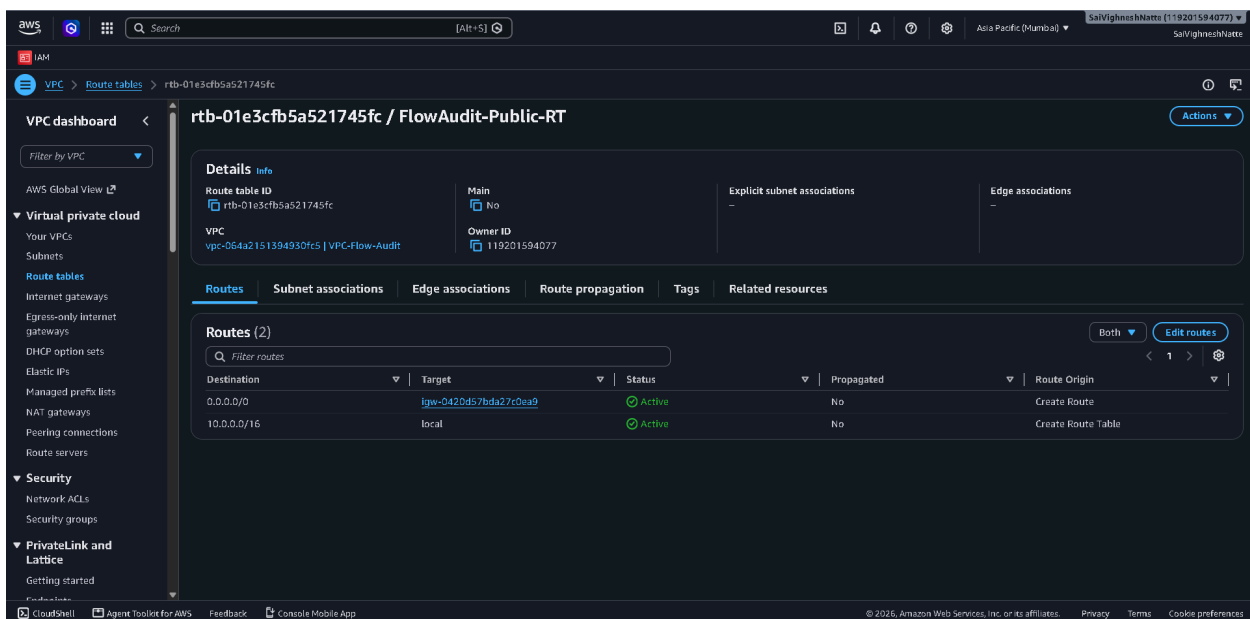
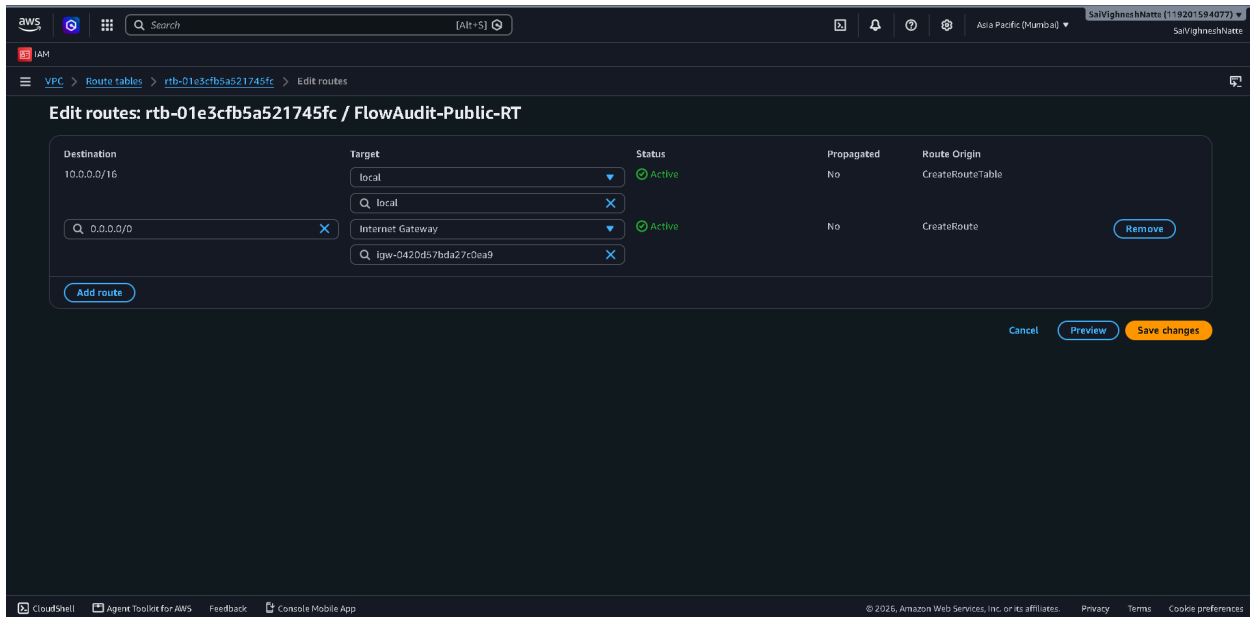
The screenshot shows the 'Attach to VPC' dialog in the AWS Management Console. The dialog prompts the user to 'Attach an internet gateway to a VPC to enable the VPC to communicate with the internet. Specify the VPC to attach below.' It lists 'Available VPCs' with a search bar containing 'vpc-064a2151394350fc5'. At the bottom, there is an 'AWS Command Line Interface command' section and two buttons: 'Cancel' and 'Attach internet gateway'.



Route Table Settings:

RT Creation-





Associate subnet with route table

VPC dashboard

Virtual private cloud

- Your VPCs
- Subnets
- Route tables**
- Internet gateways
- Egress-only internet gateways
- DHCP option sets
- Elastic IPs
- Managed prefix lists
- NAT gateways
- Peering connections
- Route servers

Security

- Network ACLs
- Security groups

PrivateLink and Lattice

- Getting started

Route tables (1/3)

Find route tables by attribute or tag

Name	Route table ID	Explicit subnet associa...	Edge associations	Main	VPC	Owner ID
-	rtb-0289d1d346cd4eb4f	-	-	Yes	vpc-02951ba6421b3fb16	119201594077
☒ FlowAudit-Public-RT	rtb-01e3cfb5a521745fc	subnet-030313cebac242a...	-	No	vpc-064a2151394930fcs VPC...	119201594077
-	rtb-045c02cf24500024d	-	-	Yes	vpc-064a2151394930fcs VPC...	119201594077

rtb-01e3cfb5a521745fc / FlowAudit-Public-RT

Details Routes Subnet associations Edge associations Route propagation Tags Related resources

Details

Route table ID rtb-01e3cfb5a521745fc	Main No	Explicit subnet associations subnet-030313cebac242ab2 / FlowAudit-Public-Subnet	Edge associations -
VPC vpc-064a2151394930fcs VPC-FlowAudit	Owner ID 119201594077		

Enable public IPv4 for subnet

VPC dashboard

Subnets (1/4)

Name	Subnet ID	State	VPC	Block Public
FlowAudit-Public-Subnet	subnet-030313cebac242ab2	Available	vpc-064a2151394930fc5 VPC...	Off
-	subnet-00c63c416bf6d40a	Available	vpc-02951ba642183fb16	Off
-	subnet-009c51b7578555b35	Available	vpc-02951ba642183fb16	Off
-	subnet-0abc6f4fa19989ac6	Available	vpc-02951ba642183fb16	Off

subnet-030313cebac242ab2 / FlowAudit-Public-Subnet

Details

Subnet ID: subnet-030313cebac242ab2

Subnet ARN: arn:aws:ec2:south-1:119201594077:subnet/subnet-030313cebac242ab2

State: Available

Block Public Access: Off

IPv4 CIDR: 10.0.1.0/24

Available IPv4 addresses: 251

IPv6 CIDR: -

VPC: vpc-064a2151394930fc5 | VPC Flow Audit

Route table: subnet-030313cebac242ab2 | FlowAudit-Public-Subnet

Edit subnet settings

Subnet

Subnet ID: subnet-030313cebac242ab2

Name: FlowAudit-Public-Subnet

Auto-assign IP settings

Enable AWS to automatically assign a public IPv4 or IPv6 address to a new primary network interface for an instance in this subnet.

☒ Enable auto-assign public IPv4 address

☐ Enable auto-assign customer-owned IPv4 address

Resource-based name (RBN) settings

Specify the hostname type for EC2 instances in this subnet and optional RBN DNS query settings.

☐ Enable resource name DNS A record on launch

☒ Enable resource name DNS AAAA record on launch

Hostname type

☐ Resource name

☒ IP name

DNS64 settings

Enable DNS64 to allow IPv6-only services in Amazon VPC to communicate with IPv4-only services and networks.

☒ Enable DNS64

EC2-create security groups

EC2 > Security Groups > Create security group

Inbound rules

Type

SSH

Protocol

TCP

Port range

22

Source

My IP

Description - optional

45.249.79.48/32

Add rule

Outbound rules

Type

All traffic

Protocol

All

Port range

All

Destination

Custom

Description - optional

0.0.0.0/0

Add rule

Rules with destination of 0.0.0.0/0 or ::/0 allow your instances to send traffic to any IPv4 or IPv6 address. We recommend setting security group rules to be more restrictive and to only allow traffic to specific known IP addresses.

Tags - optional

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

No tags associated with the resource.

Add new tag

You can add up to 50 more tags

EC2 > Security Groups > sg-0430a71a97f49b7d1

Security group (sg-0430a71a97f49b7d1 | FlowAudit-EC-SG) was created successfully

Details

sg-0430a71a97f49b7d1 - FlowAudit-EC-SG

Details

Security group name

FlowAudit-EC-SG

Security group ID

sg-0430a71a97f49b7d1

Description

Security group for VPC Flow Audit test instance

VPC ID

vpc-06da2151304930f5c

Owner

119201594077

Inbound rules count

1 Permission entry

Outbound rules count

1 Permission entry

Inbound rules

Outbound rules

Sharing

VPC associations

Related resources - new

Tags

Inbound rules (1)

Search

Name

Security group rule ID

IP version

Type

Protocol

Port range

Source

-

sg-0b1fa9d7a889a6cbf

IPv4

SSH

TCP

22

45.249.79.48/32

Amazon VPC Flow Logs Analysis for Traffic Auditing • Team-382 Team-382

Launch EC2 Instance

The screenshot shows the AWS Management Console interface. At the top, there's a navigation bar with the AWS logo, search bar, and user information. Below it, the left-hand navigation pane lists categories like Dashboard, AWS Global View, Events, Instances, Launch Templates, Spot Requests, Savings Plans, Reserved Instances, Dedicated Hosts, Capacity Reservations, UltraServers, Capacity Manager, Images, AMIs, AMI Catalog, Elastic Block Store, Volumes, Snapshots, Lifecycle Manager, and Amazon S3. The main area displays the 'Instance summary' for 'i-f0fb9c0dd1149fa10e (FlowAudit-Test-EC2)'. This summary is divided into several sections: Basic information (Instance ID, IPv6 address, Hostname type), Network settings (Public/Private IP addresses, DNS names, Elastic IP addresses), Compute settings (Auto-assigned IP address, Instance type, VPC ID, Subnet ID, Instance ARN), and Security/IAM (IAM role, IMDSv2 status, Operator). On the right side of the summary, there are buttons for 'Connect', 'Instance state', and 'Actions'. At the bottom of the console, there's a footer with links to CloudShell, Agent Toolkit for AWS, Feedback, Console Mobile App, and copyright information.

Instance summary for i-f0fb9c0dd1149fa10e (FlowAudit-Test-EC2)	
Basic Information	
Instance ID i-f0fb9c0dd1149fa10e	Public IPv4 address 43.205.126.48 open address ↗
IPv6 address -	Instance state Running
Hostname type IP name: ip-10-0-1-216.ap-south-1.compute.internal	Private IP DNS name (IPv4 only) ip-10-0-1-216.ap-south-1.compute.internal
Answer private resource DNS name -	Instance type t3.micro
Auto-assigned IP address 43.205.126.48 [Public IP]	VPC ID vpc-064a2151394930fc5 (VPC-Flow-Audit) ↗
IAM role -	Subnet ID subnet-030313cebac242ab2 (FlowAudit-Public-Subnet) ↗
IMDSv2 Required Required	Instance ARN arn:aws:ec2:ap-south-1:119201594077:instance/i-f0fb9c0dd1149fa10e
Operator -	
Additional Settings	
Elastic IP addresses -	AWS Compute Optimizer finding Opt-in to AWS Compute Optimizer for recommendations. Learn more ↗
Auto Scaling Group name -	Managed false

```
aws
[Alt+]
Search
IAM
Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023
Last login: Sat Oct 3 10:20:21 2026 from 13.233.177.5
[ec2-user@ip-10-0-1-216 ~]$ ping -c 4 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data:
64 bytes from 8.8.8.8: icmp_seq=1 ttl=117 time=2.17 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=117 time=2.13 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=117 time=2.13 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=117 time=2.13 ms
--- 8.8.8.8 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 2.125/2.136/2.165/0.016 ms
[ec2-user@ip-10-0-1-216 ~]$ curl https://example.com
```

```
aws
Search [Alt+S]
IAM
Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023
Last login: Sat Oct 3 18:20:21 2026 from 13.233.177.5
[ec2-user@ip-10-0-1-216 ~]$ ping -c 4 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=117 time=2.17 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=117 time=2.13 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=117 time=2.13 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=117 time=2.13 ms

--- 8.8.8.8 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 2.125/2.136/2.165/0.016 ms
[ec2-user@ip-10-0-1-216 ~]$ curl https://example.com
<!doctype html><html lang=en><head><meta charset=utf-8><link rel=icon href=data:><meta name=viewport content=width=device-width,initial-scale=1><title>Example Domain</title><style>html{color=scheme:light dark;background:light-dark(#eee,#222)}body{font:16px/1.6 system-ui,sans-serif;max-width:26em;margin:auto;padding:25vh 2em 2em;text-align:center}</style></head><body><p>This domain is for use in documentation examples without needing permission. This is not a service; avoid relying on it for testing and monitoring purposes.</p><script src=/s.js></script></body></html>
[ec2-user@ip-10-0-1-216 ~]$ curl https://aws.amazon.com
i-0fb9c0dd1149fa10e (FlowAudit-Test-EC2)
PublicIPs: 43.205.125.48 PrivateIPs: 10.0.1.216
CloudShell Agent Toolkit for AWS Feedback Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences
```

CloudShell + GitHub foundation :-

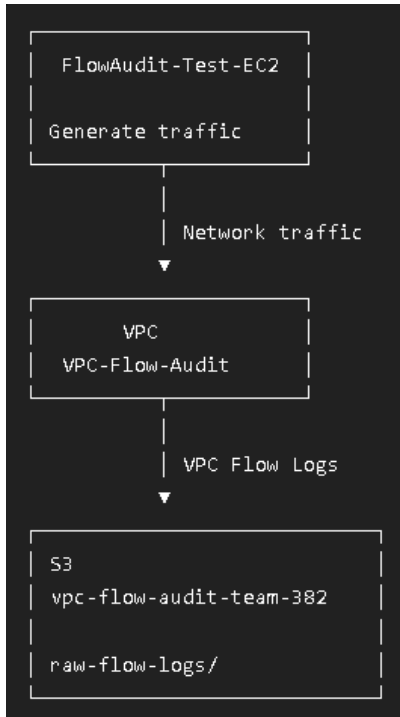
```
aws
Search [Alt+S]
CloudShell
ap-south-1
~ $ aws sts get-caller-identity
{
  "UserId": "A119201594077",
  "Account": "119201594077",
  "Arn": "arn:aws:iam::119201594077:root"
}
```

```
aws sts get-caller-identity
{
  "userId": "A119201594077",
  "accountId": "119201594077",
  "arn": "arn:aws:iam::119201594077:root"
}

$ aws configure get region
~ $ aws configure get region
~ $ aws configure set region ap-south-1
~ $
```

Phase 2

Objective:



Vpc-flow-log creation:

Successfully created flow log for the following resource:

- vpc-064a2151394930fc5

fl-0874ec1fddef6c92a

Details

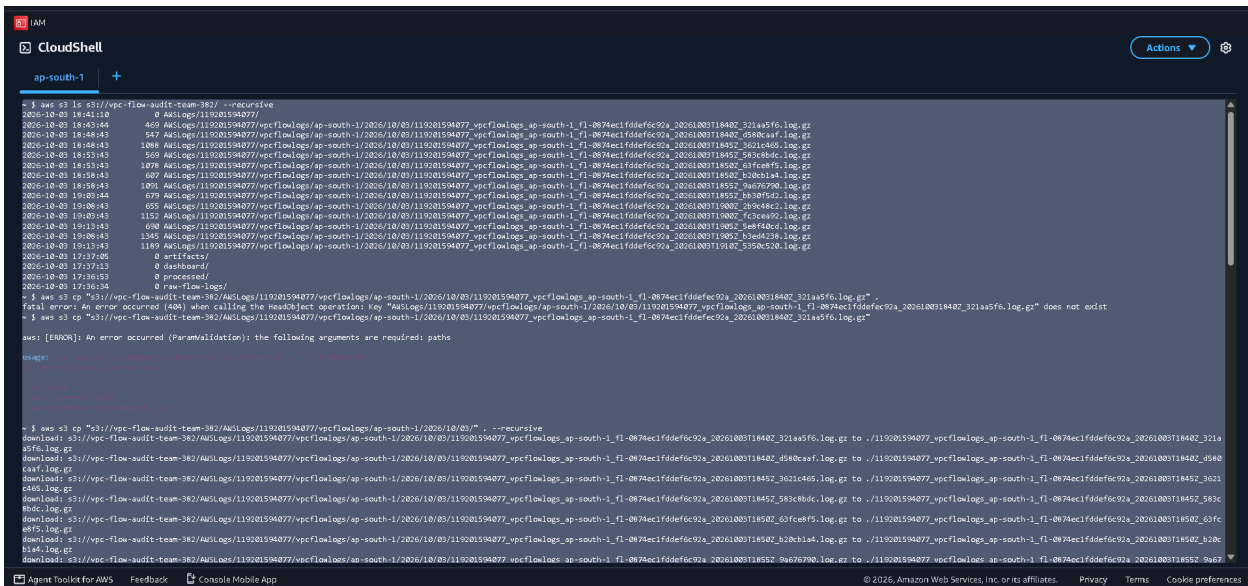
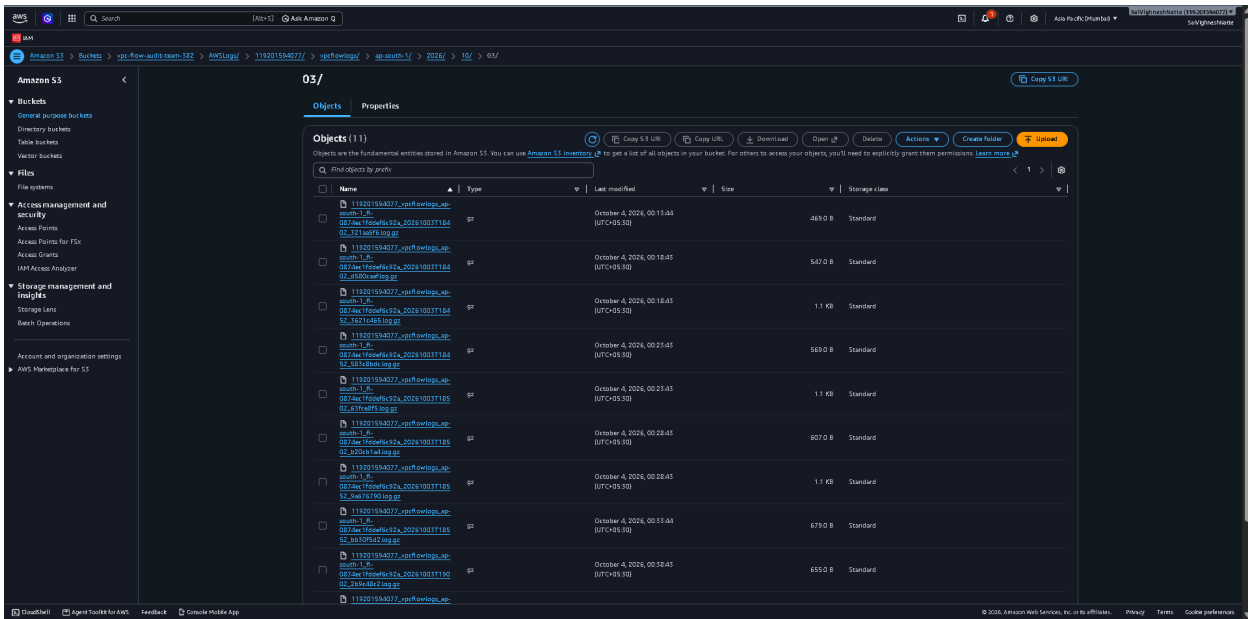
Flow Log ID fl-0874ec1fddef6c92a	Destination Type s3	Traffic Type All	File Format Plain text
Name -	Destination Name vpc-flow-audit-team-382	Max Aggregation Interval 1 minute	Hive Compatible Partitions Not enabled
State Active	IAM Role -	Log Format Default	Partition Logs Daily
Creation Time Sunday, October 4, 2026 at 00:11:09 GMT+5:30	Cross Account IAM Role -	Tag field specifications -	

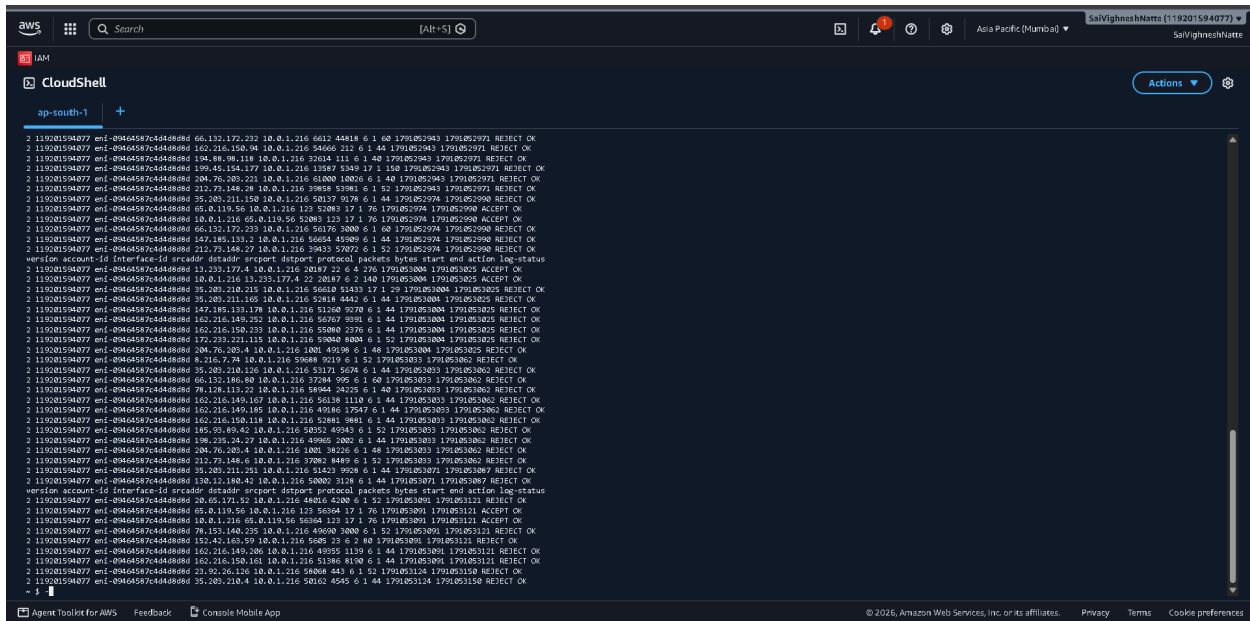
Tags

Key	Value
Project	VPCFlowAudit
Environment	Hackathon
Purpose	TrafficAuditing

i-0fb9c0dd1149fa10e (FlowAudit-Test-EC2)

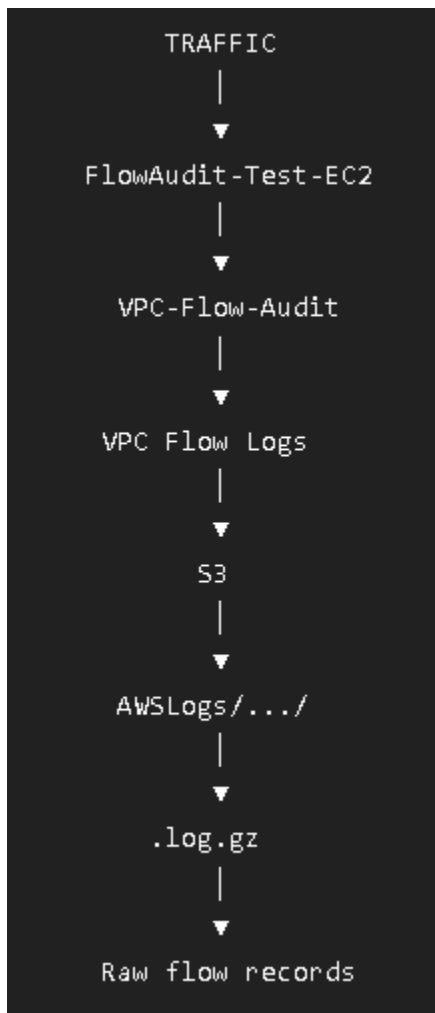
PublicIPs: 43.205.126.48 PrivateIPs: 10.0.1.216





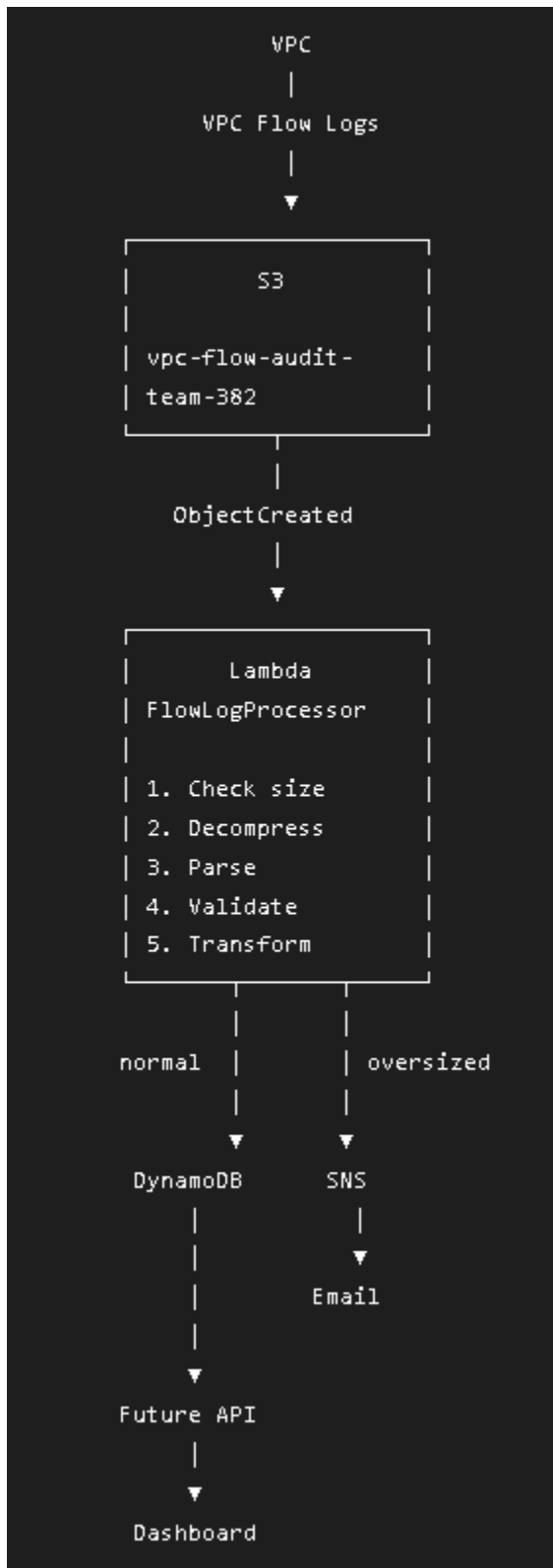
Step	Status
VPC / subnet setup	✓
Route table association	✓
EC2 instance running	✓
S3 bucket receiving VPC Flow Logs	✓
Flow Log objects created	✓
Downloaded logs from S3	✓
Decompressed and read logs	✓
Actual ACCEPT/REJECT records visible	✓

Path verified:



Phase 3 (project core)

Objective: S3 → Lambda → DynamoDB + SNS





COST CHECKPOINT #1 — Before creating anything

There are three important cost-sensitive services in this phase:

Service	Our configuration
Lambda	Small memory, short timeout
DynamoDB	Provisioned capacity, not on-demand
SNS	One email subscription

Therefore our design is:

DynamoDB:

Provisioned

5 RCU

5 WCU

Lambda:

128 MB

30 sec timeout

SNS:

1 topic

1 email subscription

No:

✗ DynamoDB on-demand

✗ DynamoDB auto scaling

✗ Provisioned concurrency

✗ Lambda provisioned concurrency

✗ Lambda VPC attachment

✗ NAT Gateway

✗ KMS customer-managed key

💰 COST CHECKPOINT #2 — Capacity mode

This is important.

Under **Table settings**, choose:

Capacity mode

Provisioned

Then:

Read capacity:

5

Write capacity:

5

Do **NOT** select:

On-demand

for this hackathon implementation.

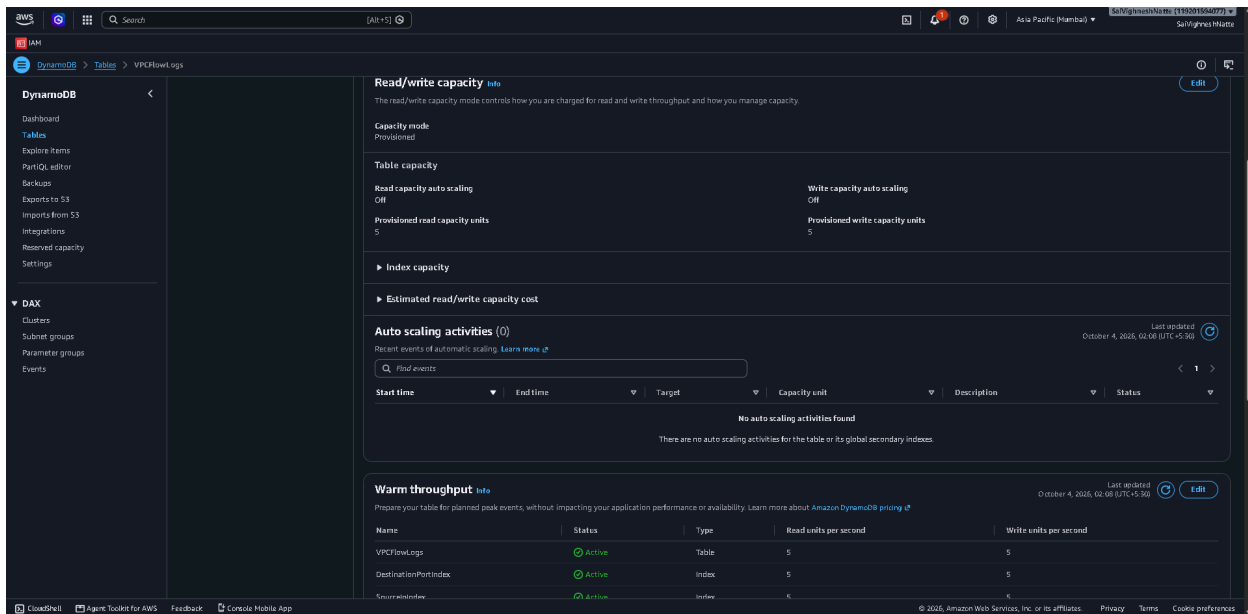
Our traffic volume is tiny, and provisioned 5/5 is comfortably within DynamoDB's 25 RCU/25 WCU always-free allowance. [AWS Document...](#)

DynamoDB Table Creation:

The screenshot shows the AWS Management Console interface. At the top, a green notification banner states: "The VPCFlowLogs table was created successfully." Below this, the "Tables (1) info" section is visible, showing a table named "VPCFlowLogs" with the following details:

Name	Status	Partition key	Sort key	Indexes	Replication Regions	Deletion protection	Favorite	Read capacity mode	Write capacity mode
VPCFlowLogs	Active	interfacelid (S)	eventKey (S)	0	0	Off	☆	Provisioned (5)	Provisioned (5)

The left sidebar shows the navigation menu with options like Dashboard, Tables, Explore Items, PartiQL editor, Backups, Exports to S3, Imports from S3, Integrations, Reserved capacity, Settings, DAX, Clusters, Subnet groups, Parameter groups, and Events. The bottom of the console shows the footer with copyright information: "© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences".



Create SNS alert topic

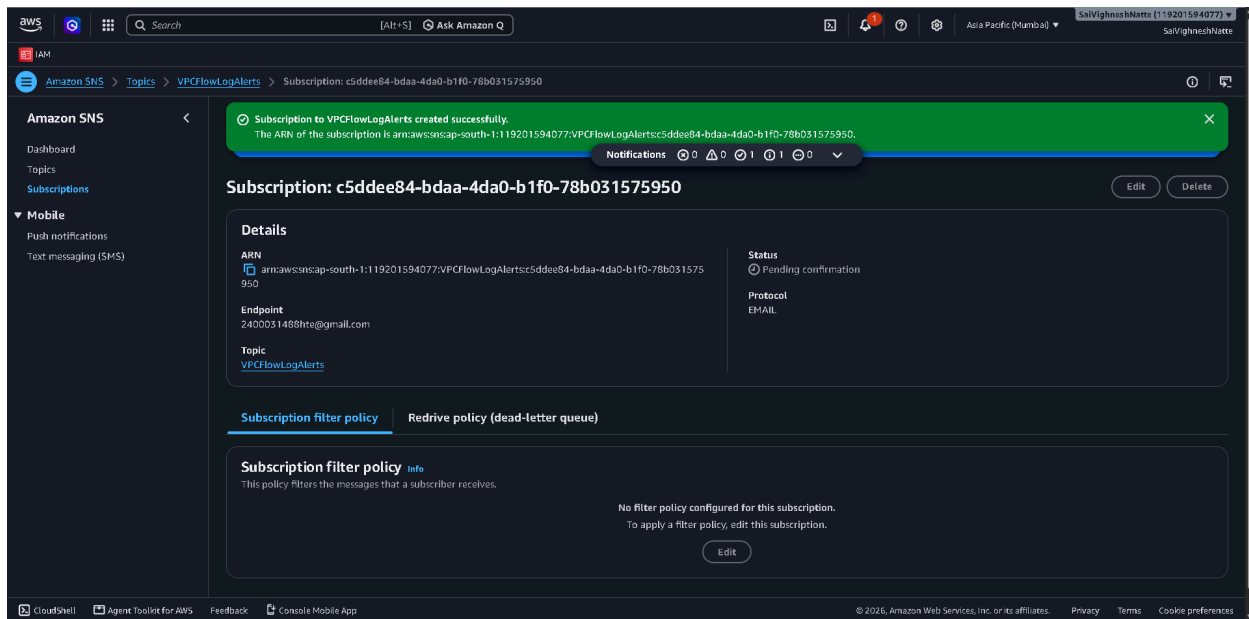
 **COST FIRST**

We only need:

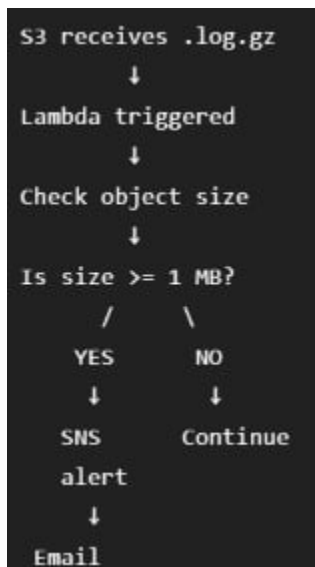
- 1 Standard SNS topic
- 1 email subscription
- No SMS
- No extra subscriptions
- No additional delivery integrations

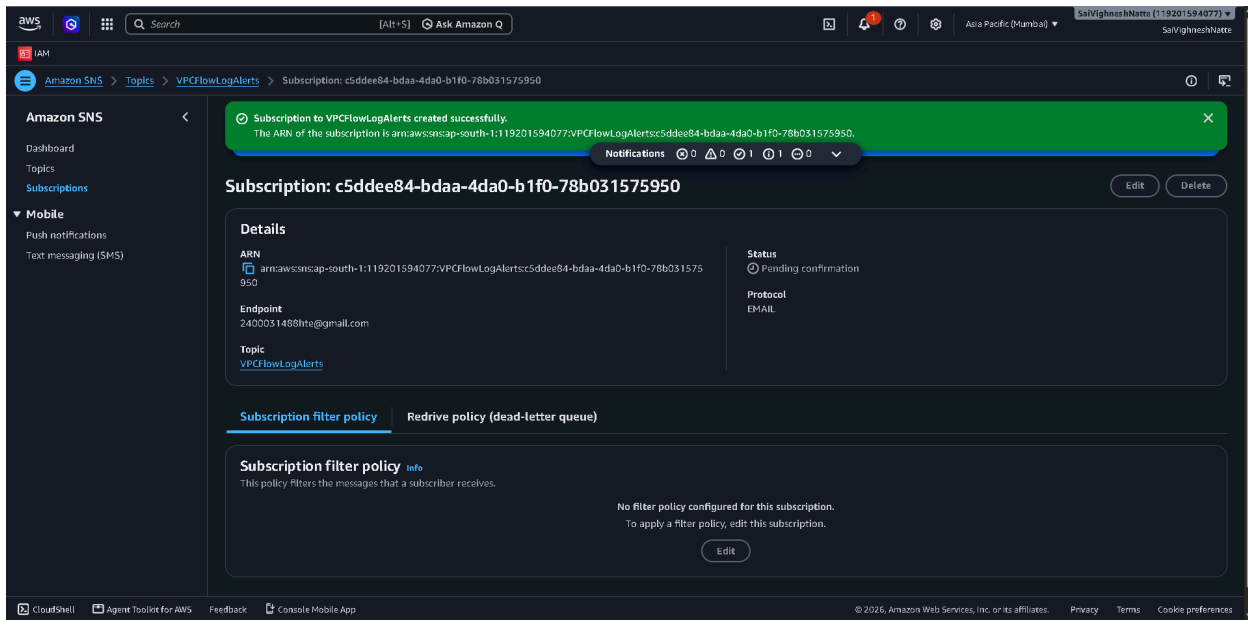


This keeps the alert mechanism simple and cost-conscious.

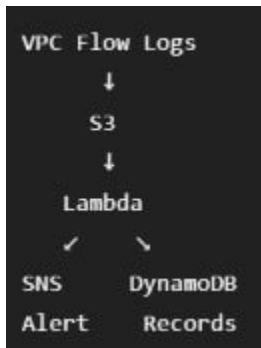


Our Lambda will check every newly-created VPC Flow Log object.
Thus, logic would be:

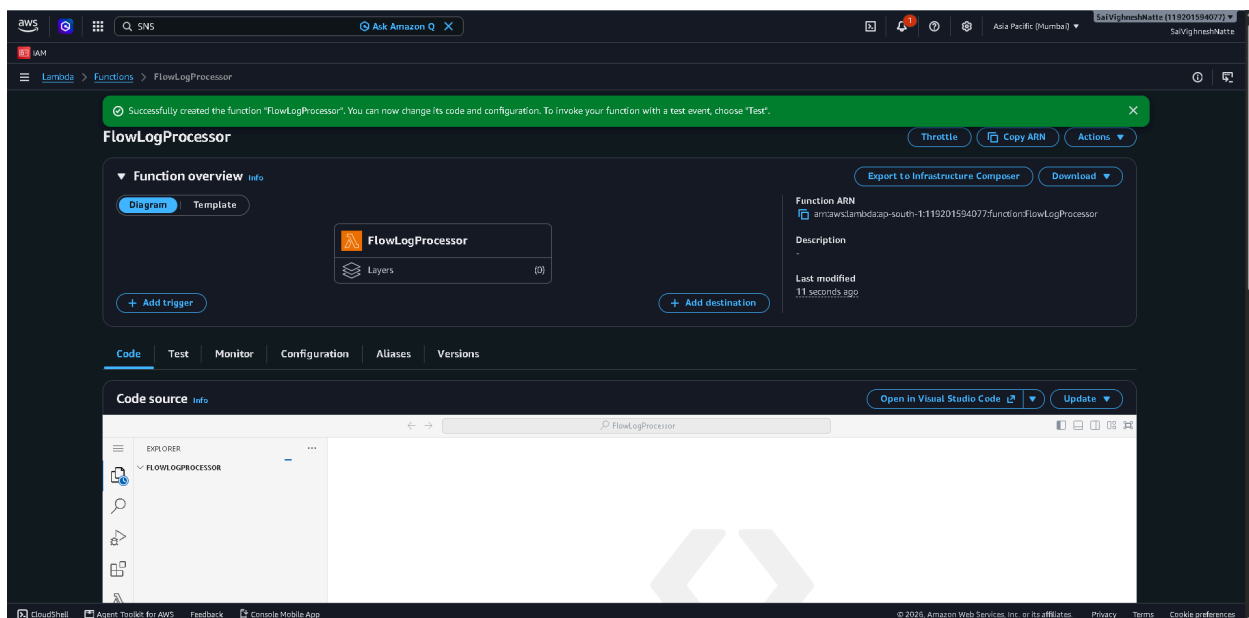




Create Lambda processor
Flow:



💰 COST CHECKPOINT — Lambda	
Before creating it, use these settings:	
Setting	Choose
Function name	FlowLogProcessor
Runtime	Python 3.14
Architecture	arm64
Memory	128 MB
Timeout	30 seconds
Provisioned concurrency	OFF
VPC	Do NOT attach
Ephemeral storage	512 MB/default



IAM Least-Privilege Permissions

Our Lambda currently has only its basic CloudWatch logging permission. We now give it exactly the permissions required by the pipeline.

Amazon VPC Flow Logs Analysis for Traffic Auditing • Team-382 Team-382



COST / SECURITY CHECK

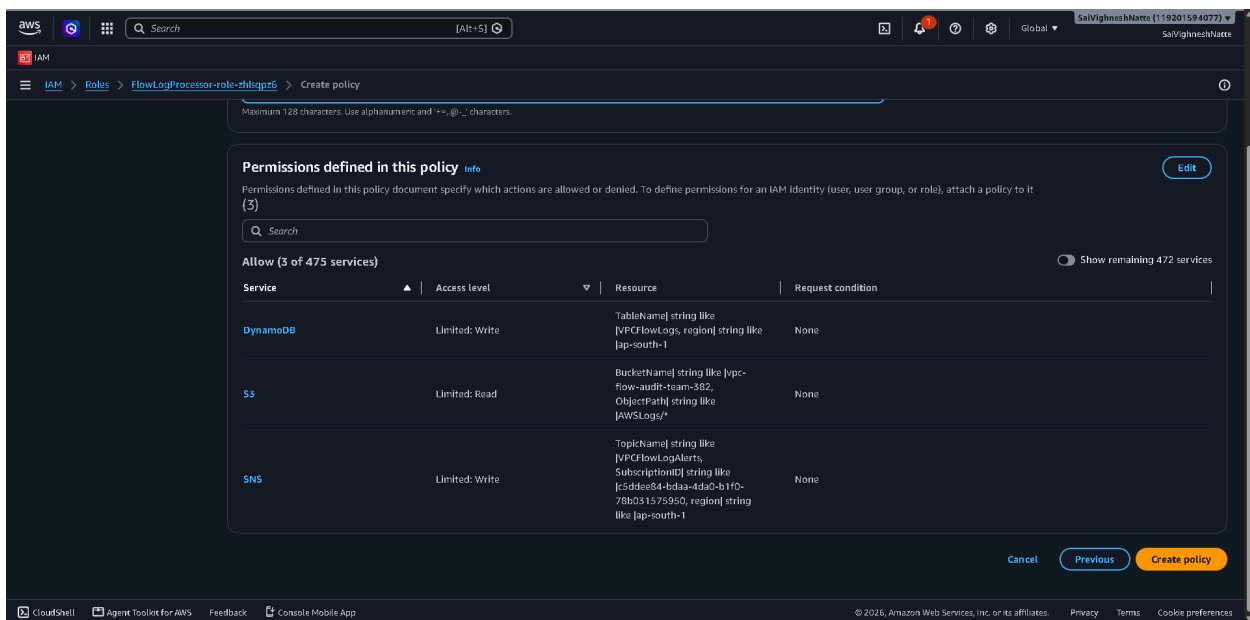
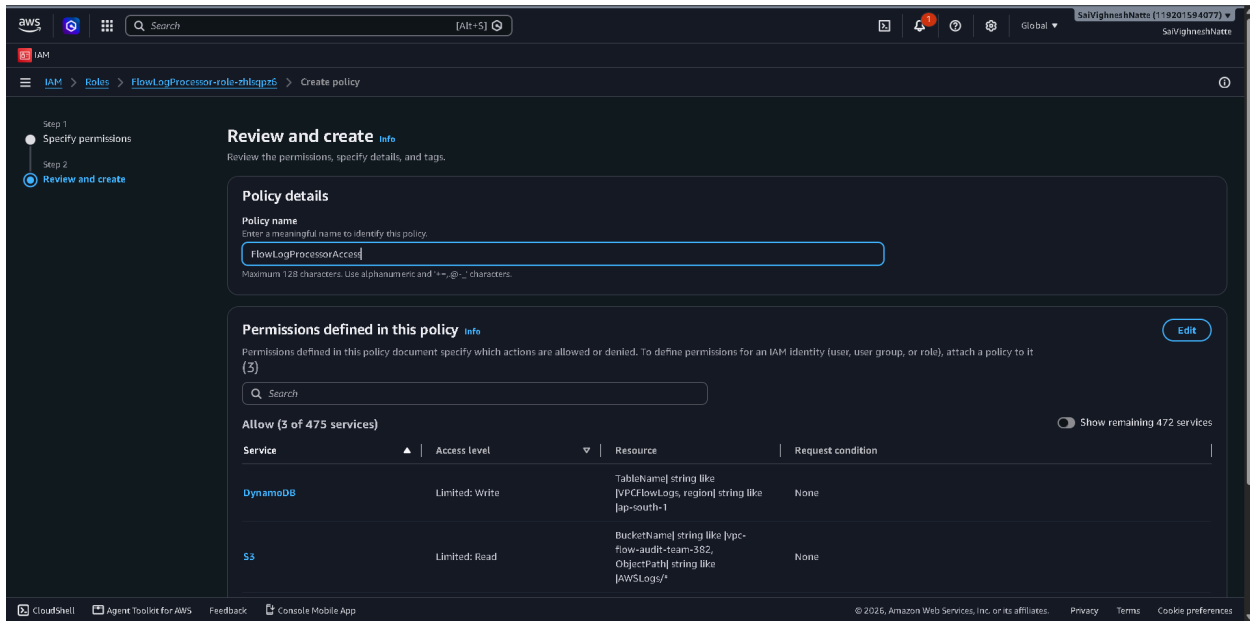
We will **not** attach:

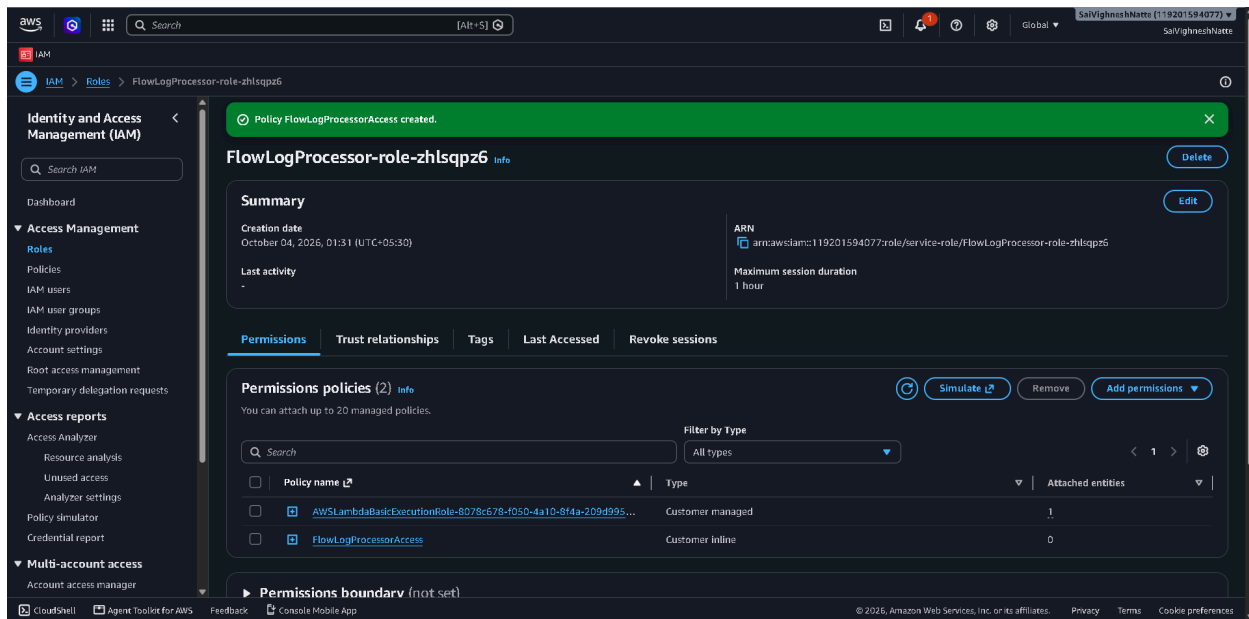
- AdministratorAccess
- AmazonS3FullAccess
- AmazonDynamoDBFullAccess
- AmazonSNSFullAccess

Instead:

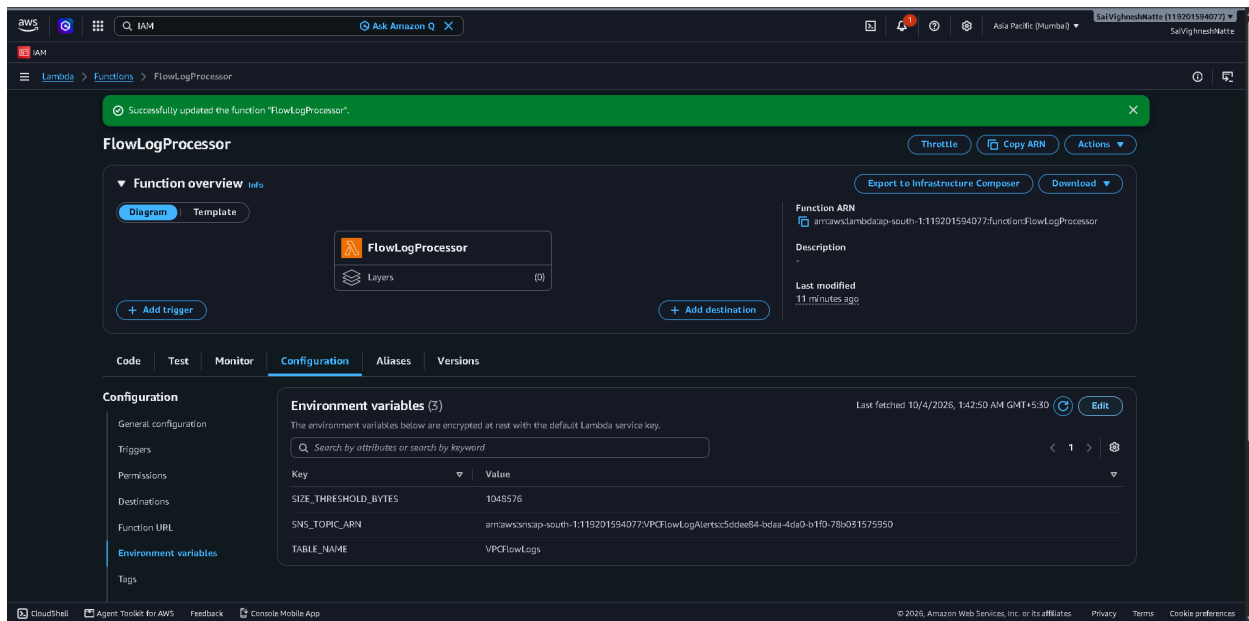
FlowLogProcessor

```
|
|— S3      → GetObject
|— DynamoDB → Put/BatchWrite
└— SNS     → Publish
```





Environment variables set:



⚠ One important DynamoDB detail

Our **DestinationPortIndex** uses:

dstPort → Number

The code handles records where AWS gives - for a port by **not creating the dstPort attribute for that record**.

That prevents the Lambda from failing with a DynamoDB type error.

The normal records will contain:

dstPort = 443

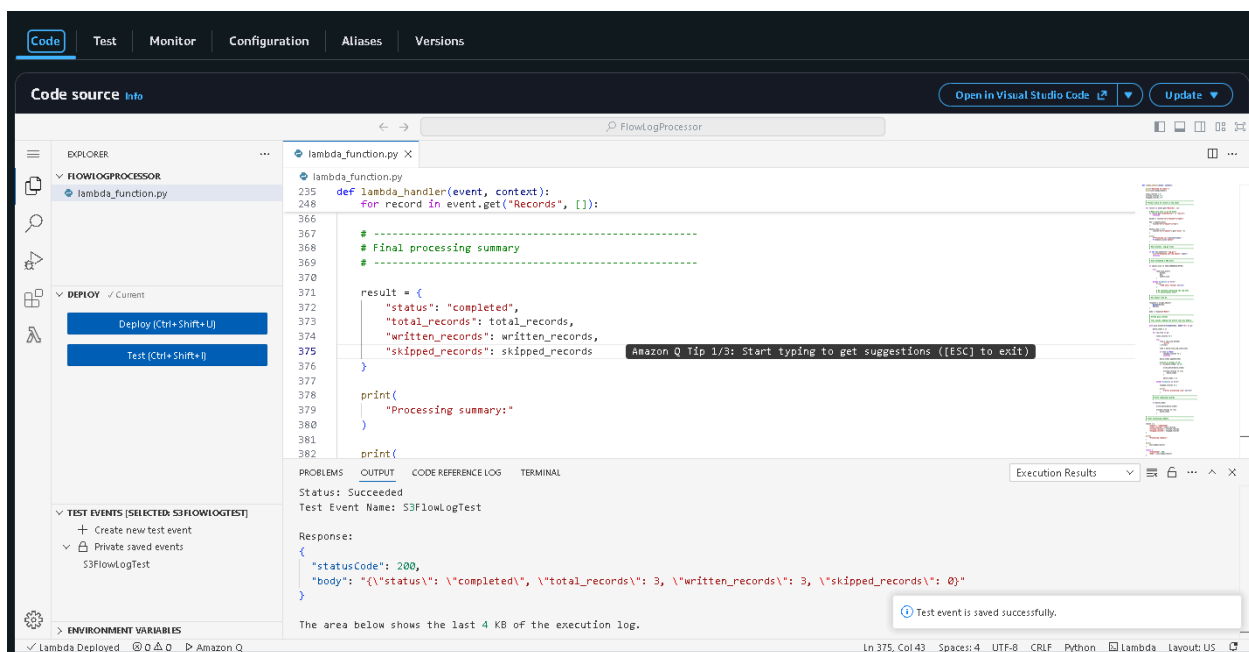
as a DynamoDB **Number**, which is exactly what our GSI expects.

The screenshot shows a VS Code editor window with the following details:

- Explorer:** Shows a file tree with `lambda_function.py` and `test_lambda_S3FlowLogTest.txt`.
- Editor:** Displays the `lambda_function.py` file. The code is a Python lambda function handler that processes records and returns a JSON response. The visible code includes:

```
376  
377  
378     result = {  
379         "status": "completed",  
380         "total_records": total_records,  
381         "written_records": written_records,  
382         "skipped_records": skipped_records  
383     }  
384  
385     print(  
386         "Processing summary:"  
387     )  
388  
389     print(  
390         json.dumps(result)  
391     )  
392  
393     return {  
394         "statusCode": 200,  
395         "body": json.dumps(result)  
396     }
```
- Terminal:** Shows the command `@lazydne369 → /workspaces/lambda_function (main) $`.

Github link: [CODE](#)



Verify DynamoDB:

The screenshot shows the AWS IAM console interface. On the left sidebar, there are navigation links for Dashboard, Tables, Explore items, PartiQL editor, Backups, Exports to S3, Imports from S3, Integrations, Reserved capacity, and Settings. The main area displays the 'VPCFlowLogs' table in DynamoDB. The table has columns: InterfaceKey (String), eventKey (String), accountId, action, bytes, dstAddr, dstPort, end, logStatus, and packets. There are three rows of data shown, each representing a different test instance.

	InterfaceKey (String)	eventKey (String)	accountId	action	bytes	dstAddr	dstPort	end	logStatus	packets
☐	eni-test123	1728000000a10.0.1.1...	123456789...	ACCEPT	800	8.8.8.8	443	1728000060	OK	10
☐	eni-test123	1728000001f410.0.1.1...	123456789...	ACCEPT	640	1.1.1.1	443	1728000061	OK	8
☐	eni-test123	1728000002a10.0.1.1...	123456789...	REJECT	300	10.0.2.20	22	1728000062	OK	5

Clean dummy data, i.e. make system ready for real data:

Search

Alt+S

Ask Amazon Q

IAM

Amazon S3

Buckets

us-east-1-us-east-1-team-382

AWSLogs/

AWSLogs/

Objects

Properties

Copy S3 URL

Copy URL

Download

Open

Delete

Actions

Create folder

Upload

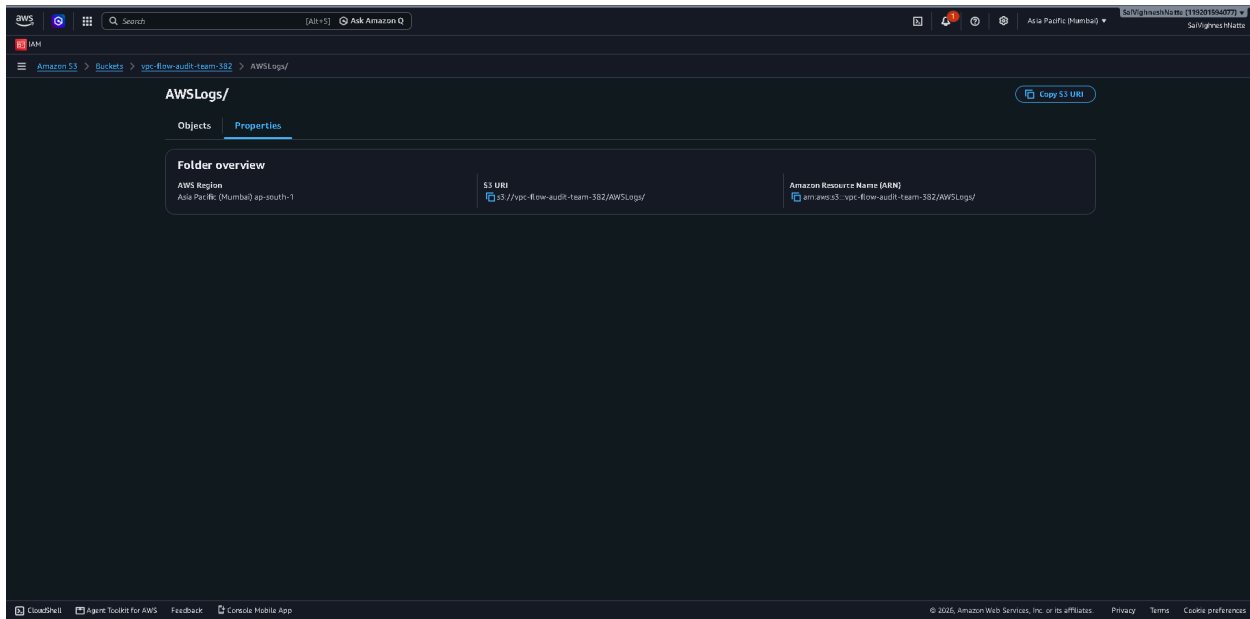
Objects (1)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 Inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

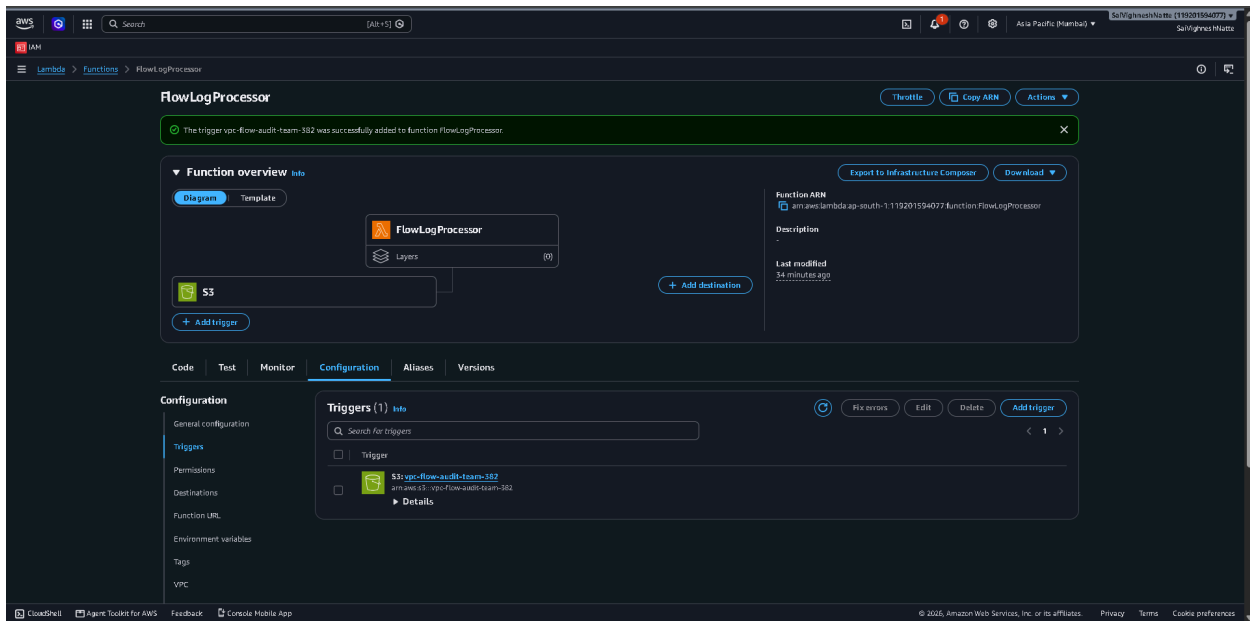
Find objects by prefix

< 1 >

Name	Type	Last modified	Size	Storage class
119201594077/	Folder	-	-	-



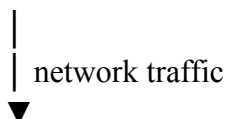
Trigger check:



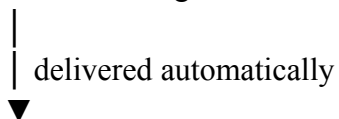
(Verification stage

)Pipeline:

EC2



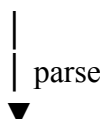
VPC Flow Logs



S3



Lambda

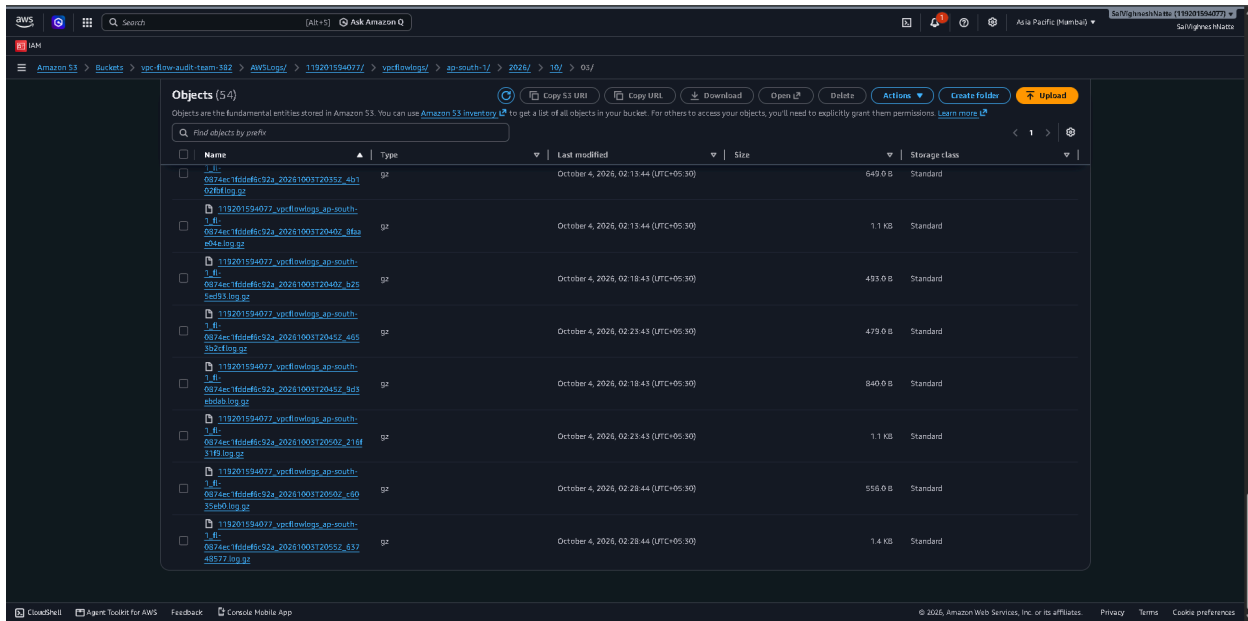


DynamoDB

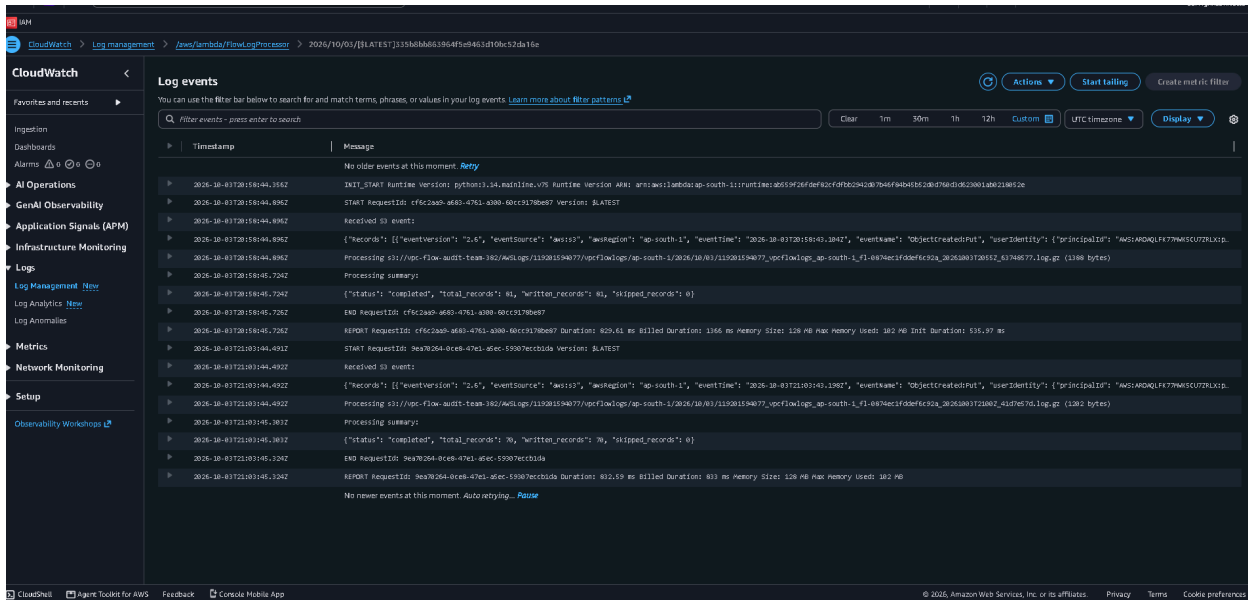
```
Amazon Linux 2023
[ec2-user@ip-10-0-1-216 ~]$ curl -I https://example.com
HTTP/2 200
date: Sat, 03 Oct 2024 20:54:59 GMT
content-type: text/html; charset=utf-8
server: cloudflare
last-modified: Fri, 02 Oct 2024 16:11:13 GMT
allow: GET, HEAD
accept-ranges: bytes
etag: 2507
cf-cache-status: HIT
x-ray: a44ed85f2bd1494-80M
alt-svc: h3=":443"; ma=66000

[ec2-user@ip-10-0-1-216 ~]$
```

i-0fb9cdd1149fa10e (FlowAudit-Test-EC2)
PublicIPs 43.205.126.48 PrivateIPs 10.0.1.216



Check Lambda accordingly using CloudWatch:
Verify the latest Lambda invocation



I can see two successful real invocations:

Invocation 1

S3 object:

.../AWSLogs/.../vpcflowlogs/ap-south-1/.../flow.log.gz

Object size:

1388 bytes

total_records:

81

written_records:

81

skipped_records:

0

Invocation 2

S3 object:

.../AWSLogs/.../vpcflowlogs/ap-south-1/.../flow.log.gz

Object size:

1202 bytes

total_records:

70

written_records:

70

skipped_records:

0

So we've successfully demonstrated:

EC2 traffic



VPC Flow Logs



S3



S3 ObjectCreated event



Lambda



81 / 70 records parsed



DynamoDB

DynamoDB:

Table: VPCFlowLogs - Items returned (50)

Scan started on October 04, 2026, 02:37:19

InterfaceId (String)	eventKey (String)	accountId	action	bytes	dstAddr	dstPort	end	logStatus	packets	protocol	srcAddr	srcPort
eni-09464587c4d4d8dbd	1751060681852.203...	119201594...	REJECT	44	10.0.1.216	8723	1751060710	OK	1	6	216.25.83.144	54227
eni-09464587c4d4d8dbd	1751060681852.203...	119201594...	REJECT	44	10.0.1.216	8061	1751060710	OK	1	6	35.203.210...	55400
eni-09464587c4d4d8dbd	1751060681852.203...	119201594...	REJECT	44	10.0.1.216	11122	1751060710	OK	1	6	35.203.210...	50422
eni-09464587c4d4d8dbd	17510607128157.245...	119201594...	REJECT	40	10.0.1.216	6638	1751060741	OK	1	6	157.245.10...	53437
eni-09464587c4d4d8dbd	17510607128205.210...	119201594...	REJECT	44	10.0.1.216	8082	1751060741	OK	1	6	205.210.31.79	53143
eni-09464587c4d4d8dbd	17510607128...	119201594...	REJECT	52	10.0.1.216	31278	1751060741	OK	1	6	212.73.148.22	52322
eni-09464587c4d4d8dbd	17510607128212.73...	119201594...	REJECT	52	10.0.1.216	6666	1751060741	OK	1	6	212.73.148.28	54342
eni-09464587c4d4d8dbd	1751060712823.52.2...	119201594...	REJECT	52	10.0.1.216	1724	1751060741	OK	1	6	23.92.26.77	38220
eni-09464587c4d4d8dbd	1751060712835.203...	119201594...	REJECT	44	10.0.1.216	5936	1751060741	OK	1	6	35.203.211.13	53903
eni-09464587c4d4d8dbd	1751060712835.203...	119201594...	REJECT	44	10.0.1.216	8153	1751060741	OK	1	6	35.203.211.91	57275
eni-09464587c4d4d8dbd	1751060712866.132...	119201594...	REJECT	60	10.0.1.216	2380	1751060741	OK	1	6	66.132.186.46	2069
eni-09464587c4d4d8dbd	1751060712869.37.1...	119201594...	REJECT	40	10.0.1.216	9570	1751060741	OK	1	6	89.37.172.141	36184
eni-09464587c4d4d8dbd	1751060713810.0.1.2...	119201594...	ACCEPT	232	13.200.90.133	443	1751060768	OK	4	6	10.0.1.216	53460
eni-09464587c4d4d8dbd	1751060713813.200...	119201594...	ACCEPT	232	10.0.1.216	53480	1751060768	OK	4	6	13.200.90.133	443
eni-09464587c4d4d8dbd	17510607138152.42...	119201594...	REJECT	40	10.0.1.216	21	1751060768	OK	1	6	152.42.183.59	5805
eni-09464587c4d4d8dbd	17510607138162.243...	119201594...	REJECT	40	10.0.1.216	22	1751060768	OK	1	6	162.245.17...	42492
eni-09464587c4d4d8dbd	17510607138165.154...	119201594...	REJECT	44	10.0.1.216	5969	1751060768	OK	1	6	165.154.36...	43450
eni-09464587c4d4d8dbd	17510607138198.235...	119201594...	REJECT	44	10.0.1.216	2087	1751060768	OK	1	6	198.235.24...	53153

For checking sns updated threshold to 1:

Successfully updated the function "FlowLogProcessor"

FlowLogProcessor

Function overview

Function ARN: arn:aws:lambda:ap-south-1:119201594077:function:FlowLogProcessor

Description: -

Last modified: 2 minutes ago

Configuration

Environment variables (3)

The environment variables below are encrypted at rest with the default Lambda service key.

Key	Value
SIZE_THRESHOLD_BYTES	1
SNS_TOPIC_ARN	arn:aws:sns:ap-south-1:119201594077:VPCFlowLogAlerts:C5ddee84-bdae-46a0-b180-78b031575950
TABLE_NAME	VPCFlowLogs

```
~ $ ls -lh test-flow.log.gz
-rw-r--r-- 1 cloudshell-user cloudshell-user 158 Oct 3 2019 test-flow.log.gz
~ $ aws s3 cp test-flow.log.gz s3://pc-flow-audit-team-382/awlogs/test-flow.log.gz
upload ./test-flow.log.gz to s3://pc-flow-audit-team-382/awlogs/test-flow.log.gz
~ $ aws sns list-topics --region ap-south-1
{
  "Topics": [
    {
      "TopicArn": "arn:aws:sns:ap-south-1:1119201590771:PCFlowLogAlerts"
    }
  ]
}
~ $ aws sns get-topic-attributes \
> --topic-arn "arn:aws:sns:ap-south-1:1119201590771:PCFlowLogAlerts" \
> --region ap-south-1
{
  "Attributes": {
    "Policy": "{\"version\":\"2008-10-17\",\"id\":\"_default_policy_ID\",\"statement\":[{\"sid\":\"_default_statement_ID\",\"effect\":\"allow\",\"principal\":{\"AWS\":\"*\"},\"action\":[\"sns:Publish\",\"sns:RemovePermission\",\"sns:SetTopicAttributes\",\"sns:DeleteTopic\",\"sns:ListSubscriptionsByTopic\"],\"resource\":\"arn:aws:sns:ap-south-1:1119201590771:PCFlowLogAlerts\",\"condition\":{\"StringEquals\":{\"AWS:SourceAccount\":\"1119201590771\"}}}],\"subscriptionsPending\": \"0\", \"TopicArn\": \"arn:aws:sns:ap-south-1:1119201590771:PCFlowLogAlerts\", \"TracingConfig\": \"PassThrough\", \"EffectiveLabelingPolicy\": {\"http\":{\"defaultHealthyRetryPolicy\":{\"backoffTarget\":\"120\",\"numRetries\":\"3\",\"numDelayRetries\":\"0\",\"numDelayRetries\":\"0\",\"backoffFunction\":\"Linear\"},\"disableSubscriptionOverrides\":false,\"defaultRequestPolicy\":{\"headerContentType\":\"text/plain; charset=UTF-8\"}}, \"redeliveryAgility\": \"None\", \"subscriptionsConfirmed\": \"1\", \"labelingPolicy\": \"None\", \"subscriptionsDeleted\": \"0\" }",
    "TopicName": "PCFlowLogAlerts"
  }
}
~ $
```

Here the event is triggered

S3



Lambda



158 bytes >= 1 byte

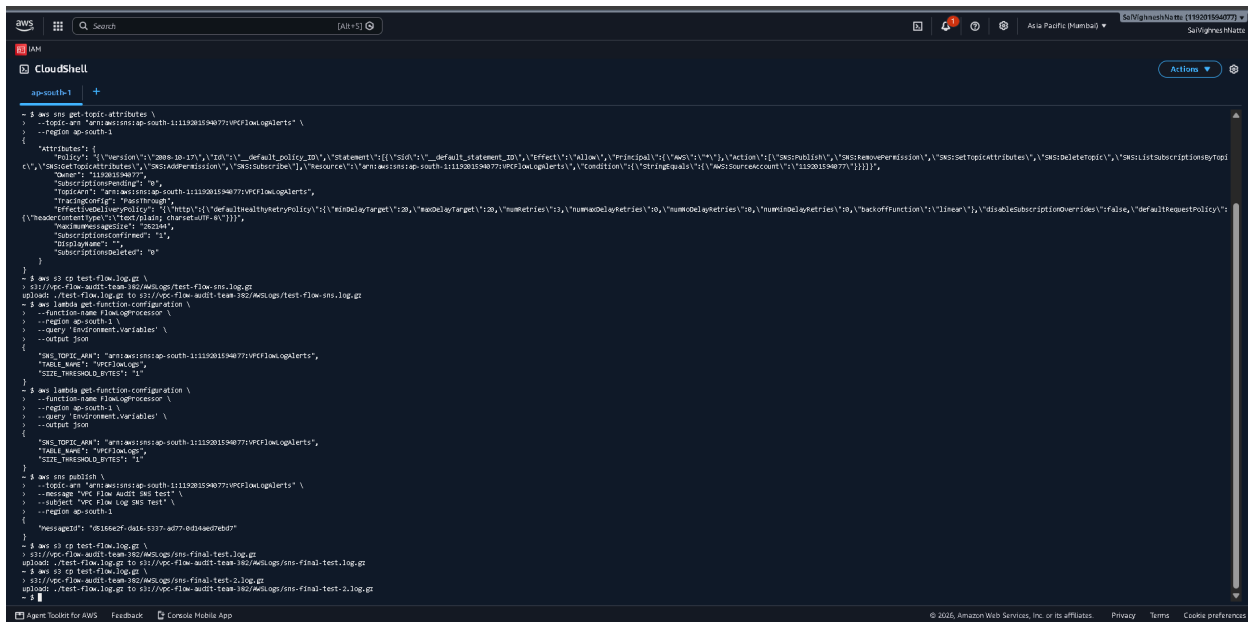
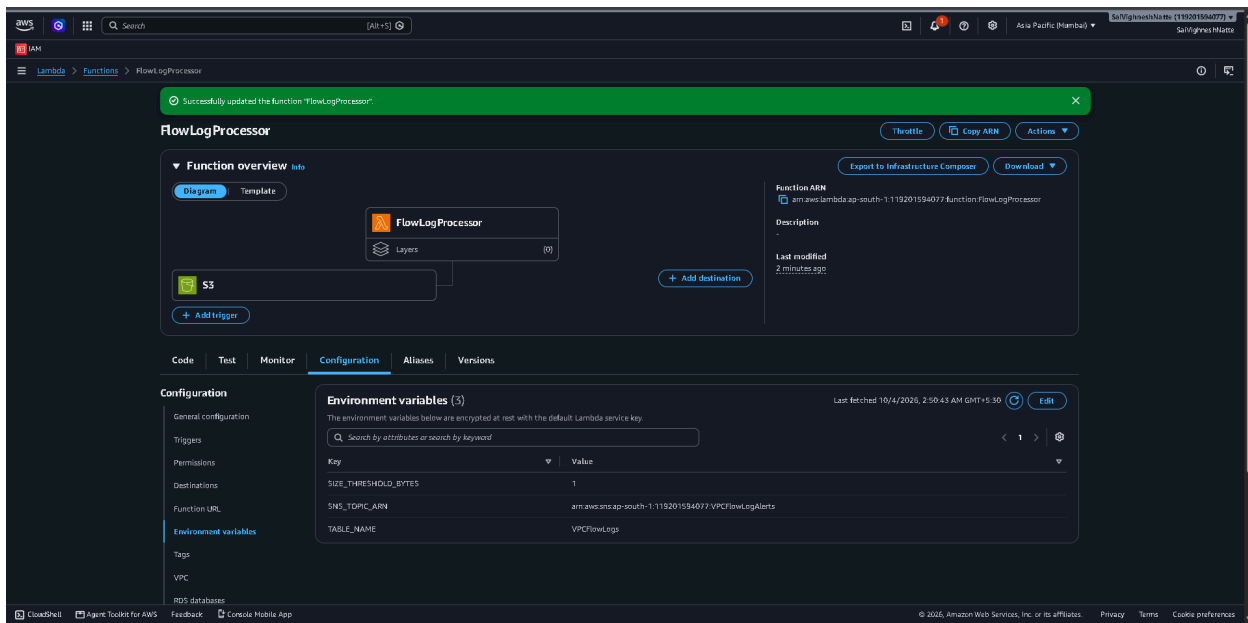


SNS

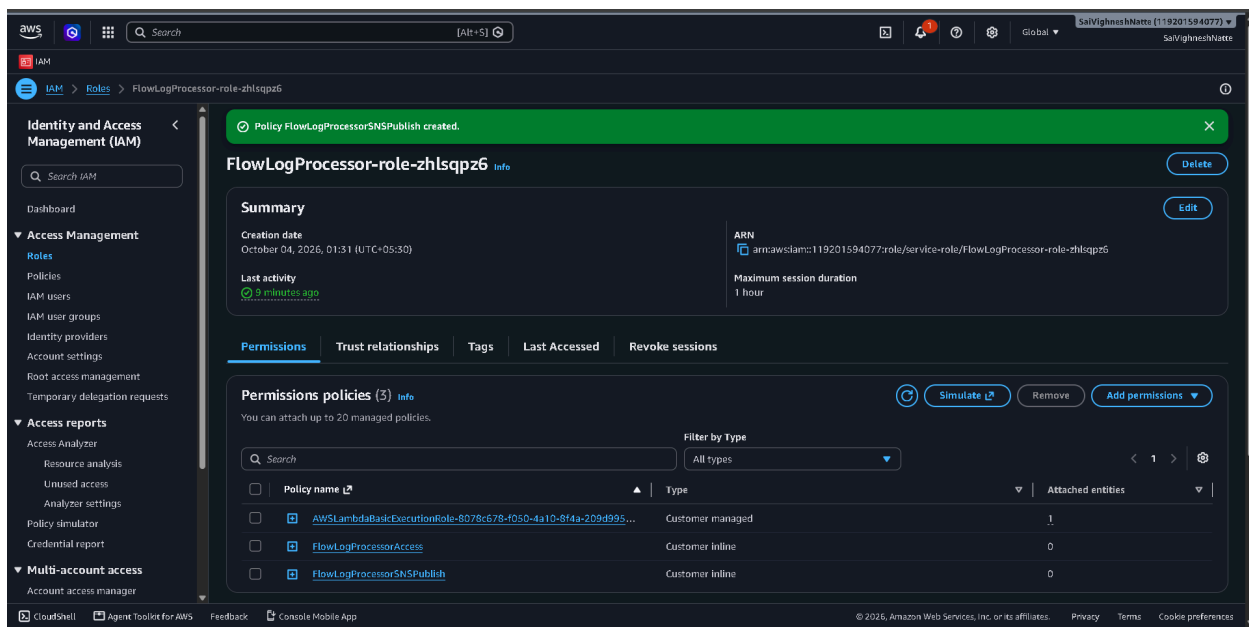


Email

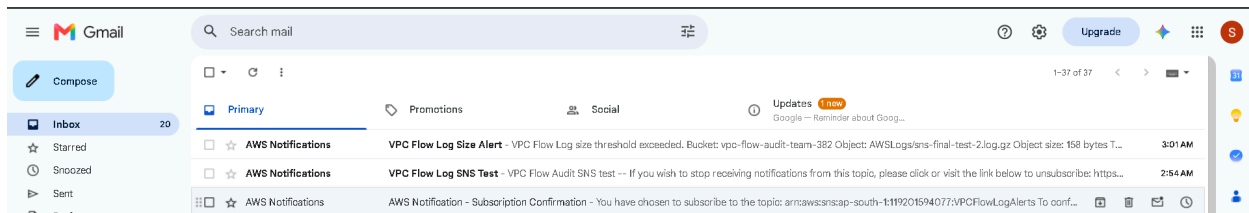
```
~ $ ls -lh test-flow.log.gz
-rw-r--r-- 1 cloudshell-user cloudshell-user 158 Oct 3 2019 test-flow.log.gz
~ $ aws s3 cp test-flow.log.gz s3://pc-flow-audit-team-382/awlogs/test-flow.log.gz
upload ./test-flow.log.gz to s3://pc-flow-audit-team-382/awlogs/test-flow.log.gz
~ $ aws sns list-topics --region ap-south-1
{
  "Topics": [
    {
      "TopicArn": "arn:aws:sns:ap-south-1:1119201590771:PCFlowLogAlerts"
    }
  ]
}
~ $ aws sns get-topic-attributes \
> --topic-arn "arn:aws:sns:ap-south-1:1119201590771:PCFlowLogAlerts" \
> --region ap-south-1
{
  "Attributes": {
    "Policy": "{\"version\":\"2008-10-17\",\"id\":\"_default_policy_ID\",\"statement\":[{\"sid\":\"_default_statement_ID\",\"effect\":\"allow\",\"principal\":{\"AWS\":\"*\"},\"action\":[\"sns:Publish\",\"sns:RemovePermission\",\"sns:SetTopicAttributes\",\"sns:DeleteTopic\",\"sns:ListSubscriptionsByTopic\"],\"resource\":\"arn:aws:sns:ap-south-1:1119201590771:PCFlowLogAlerts\",\"condition\":{\"StringEquals\":{\"AWS:SourceAccount\":\"1119201590771\"}}}],\"subscriptionsPending\": \"0\", \"TopicArn\": \"arn:aws:sns:ap-south-1:1119201590771:PCFlowLogAlerts\", \"TracingConfig\": \"PassThrough\", \"EffectiveLabelingPolicy\": {\"http\":{\"defaultHealthyRetryPolicy\":{\"backoffTarget\":\"120\",\"numRetries\":\"3\",\"numDelayRetries\":\"0\",\"numDelayRetries\":\"0\",\"backoffFunction\":\"Linear\"},\"disableSubscriptionOverrides\":false,\"defaultRequestPolicy\":{\"headerContentType\":\"text/plain; charset=UTF-8\"}}, \"redeliveryAgility\": \"None\", \"subscriptionsConfirmed\": \"1\", \"labelingPolicy\": \"None\", \"subscriptionsDeleted\": \"0\" }",
    "TopicName": "PCFlowLogAlerts"
  }
}
~ $
```



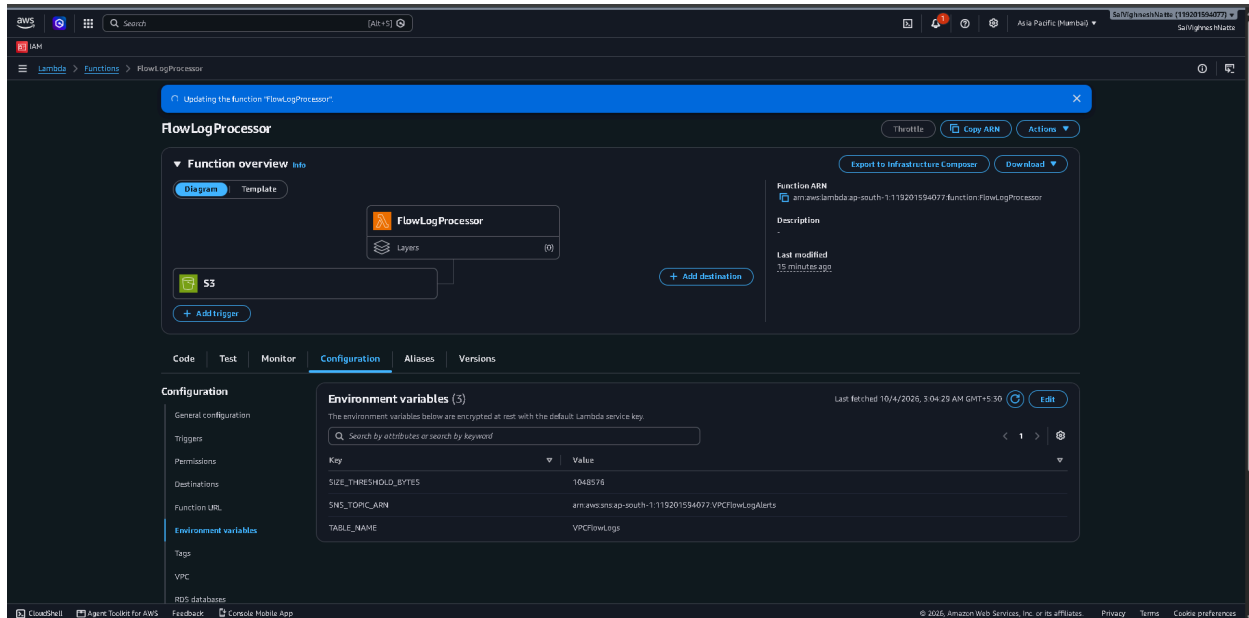
Faced many errors with SNS testing from triggering to email invalidation, at last the solution was a new IAM role:



SNS TESTING - SUCCESSFUL:



As testing is completed, restore the real threshold



Backend pipeline	
Component	Status
VPC Flow Logs	✓
S3 storage	✓
S3 → Lambda trigger	✓
Lambda parsing	✓
Lambda → DynamoDB	✓
DynamoDB GSIs	✓
Lambda → SNS	✓
SNS → Email	✓
Least-privilege SNS permission	✓

Phase 1

Objective:

User / Dashboard



API Gateway



Query Lambda

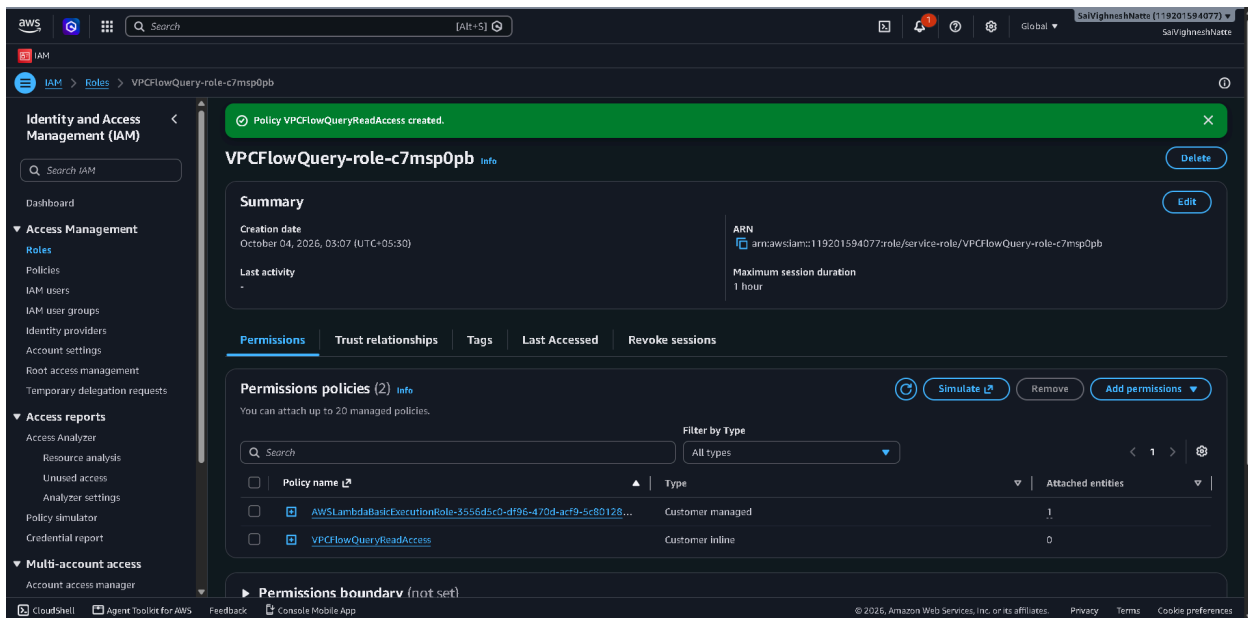
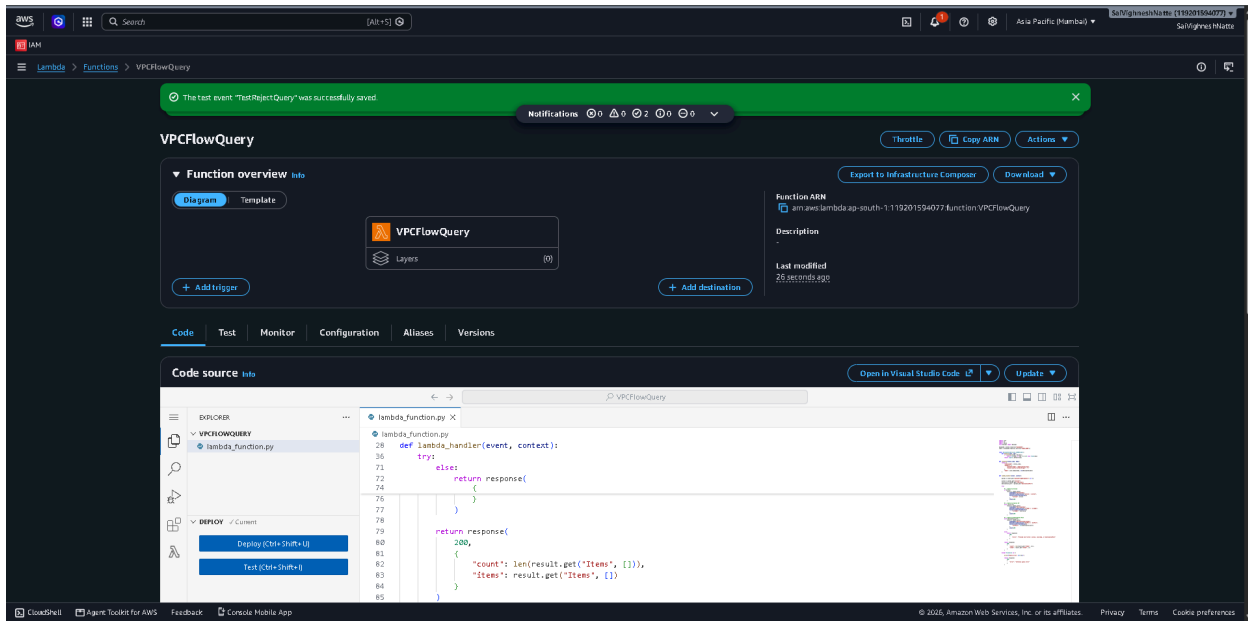


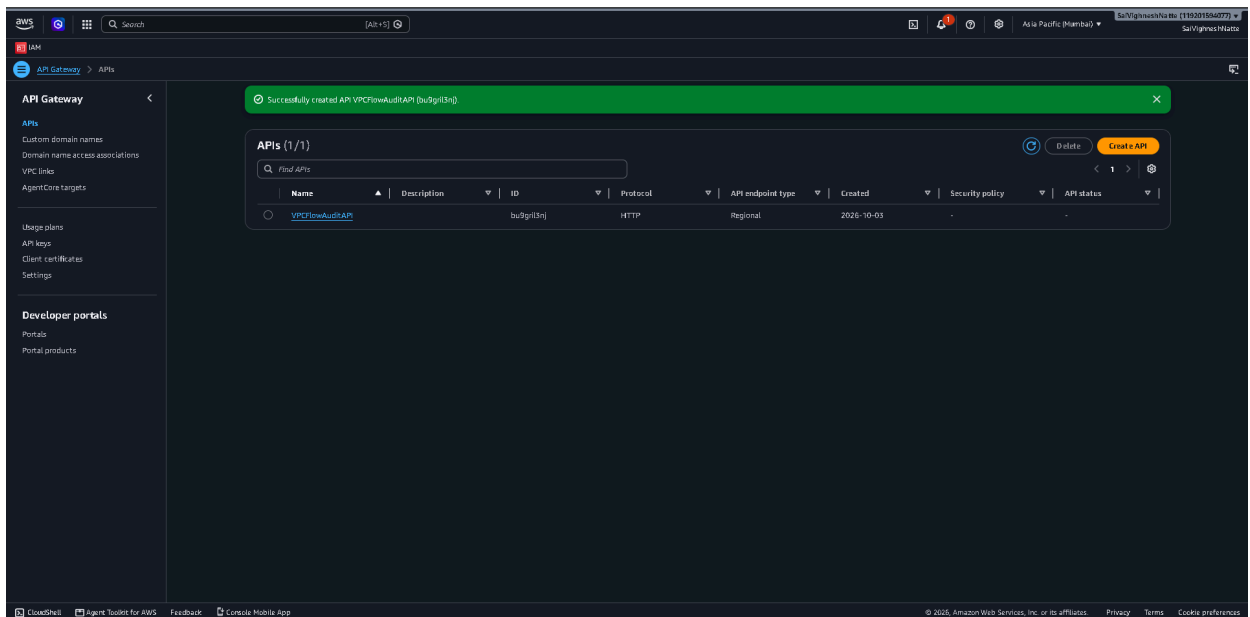
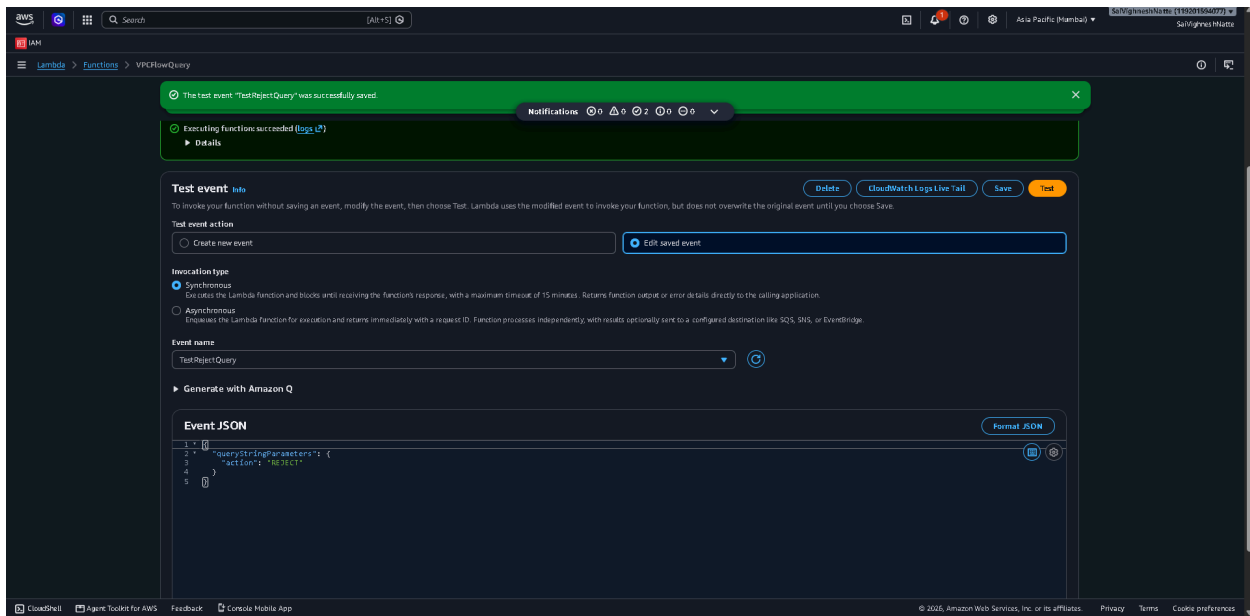
DynamoDB



Action SourceIP DestinationPort

Index Index Index





API Gateway > APIs > VPCFlowAuditAPI (bu3grl3n) > Stages

API Gateway

- APIs
- Custom domain names
- Domain name access associations
- VPC links

API: VPCFlowAuditAPI (bu3grl3n)

Develop

- Routes
- Authorization
- Integrations
- CORS
- Reimport
- Export

Deploy

- Stages

Monitor

- Metrics
- Logging

Protect

- Throttling

Successfully created API VPCFlowAuditAPI (bu3grl3n)

Stages

Stages for VPCFlowAuditAPI

Search

- \$default

Create

Stage details

Details

Name	Default	Created	October 4, 2026 3:20 AM	Last updated	October 4, 2026 3:20 AM
Invoke URL	https://bu3grl3n.execute-api-south-1.amazonaws.com				
Description	None				

Attached deployment

Automatic Deployment

Enabled

Deployment ID	Deployment created
azh561	October 4, 2026 3:20 AM

Deployment description

Automatic deployment triggered by changes to the Api configuration

Stage variables

Find resources

Key	Value
No Stage Variables	

You don't have any stage variables

Edit

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

```
pretty-print
{"error": "Internal query error"}
```

Limits are changed

[illegible]

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Action": "iam:CreateUser",
      "Resource": "arn:aws:iam::111111111111:role/*",
      "Effect": "Allow"
    },
    {
      "Action": "iam:CreateRole",
      "Resource": "arn:aws:iam::111111111111:role/*",
      "Effect": "Allow"
    }
  ]
}

```

```
aws
[Alt+F5]
CloudShell
ap-south-1 x ap-south-1 +
{
  "eventKey": {
    "S1": "1720800002018.0.1.10#1.1.1.10#41"
  },
  "version": {
    "M": "1"
  },
  "action": {
    "S1": "REJECT"
  },
  "accountId": {
    "S1": "123456789012"
  },
  "tagSet": {
    "S1": "tag"
  },
  "srcPort": {
    "M": "54321"
  },
  "dstPort": {
    "M": "123"
  },
  "interfaceId": {
    "S1": "eni-test123"
  },
  "protocol": {
    "M": "6"
  },
  "bytes": {
    "M": "300"
  },
  "direction": {
    "S1": "0.0.2.20"
  },
  "end": {
    "M": "1720800042"
  },
  "srcAddr": {
    "S1": "20.0.1.10"
  },
  "start": {
    "M": "1720800002"
  },
  "eventKey": {
    "S1": "1720800002018.0.1.10#10.0.1.20#21"
  }
},
{
  "count": 1,
  "sampleCount": 1,
  "consumedCapacity": null
}
waiting for data... (interrupt to abort)
```

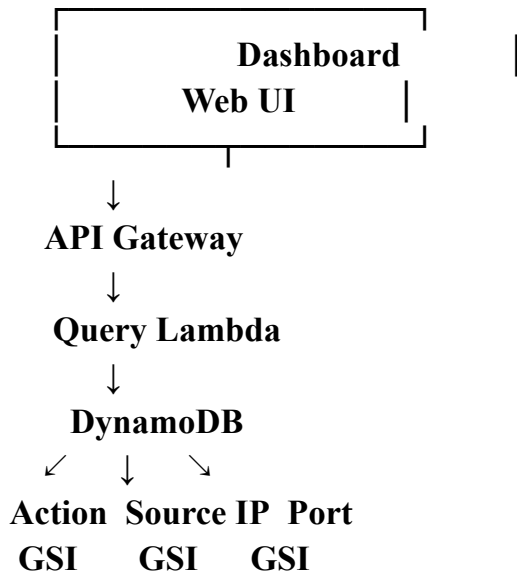
```
aws
[Alt+F5]
CloudShell
ap-south-1 +
- $ aws dynamodb query \
> --table-name 'CPFlowLog' \
> --key-condition-expression 'interfaceId = :id' \
> --expression-attribute-values '{":id":{"S":"eni-test123"}}' \
> --select 'count' \
> --region ap-south-1
{
  "count": 1,
  "sampleCount": 1,
  "consumedCapacity": null
}
- $
```

```
aws dynamodb query \
--table-name vpcflowlogs \
--key-condition-expression "interfaceid = i1d" \
--expression-attribute-values {'i1d':{'S':'eni-test123'}} \
--select count \
--region ap-south-1
{
  "Count": 3,
  "ScannedCount": 3,
  "ConsumedCapacity": null
}
aws dynamodb query \
--table-name vpcflowlogs \
--key-condition-expression "interfaceid = i1d" \
--expression-attribute-values {'i1d':{'S':'eni-test123'}} \
--projection-expression "interfaceid,sourceip" \
--region ap-south-1 \
--output json > raw-test-items.json
$ python3 -c 'P'
import json
import subprocess

with open("/tmp/test-items.json") as f:
    data = json.load(f)
    items = data.get("Items", [])
    for item in items:
        key = {
            "interfaceid": item["interfaceid"],
            "sourceip": item["sourceip"]
        }
        subprocess.run([
            "aws", "dynamodb", "delete-item",
            "--table-name", "vpcflowlogs",
            "--key", json.dumps(key),
            "--region", "ap-south-1"
        ], check=True)
    print(f"deleted {len(items)} synthetic records.")
P
deleted 3 synthetic records.
aws dynamodb query \
--table-name vpcflowlogs \
--key-condition-expression "interfaceid = i1d" \
--expression-attribute-values {'i1d':{'S':'eni-test123'}} \
--select count \
--region ap-south-1
{
  "Count": 0,
  "ScannedCount": 0,
  "ConsumedCapacity": null
}
$
```

PHASE 5 — Dashboard

Our target is a simple, lightweight dashboard that demonstrates the actual value of the project:



Dashboard should show

1. Summary

- **Total records returned**
- **ACCEPT count**
- **REJECT count**

2. Filters

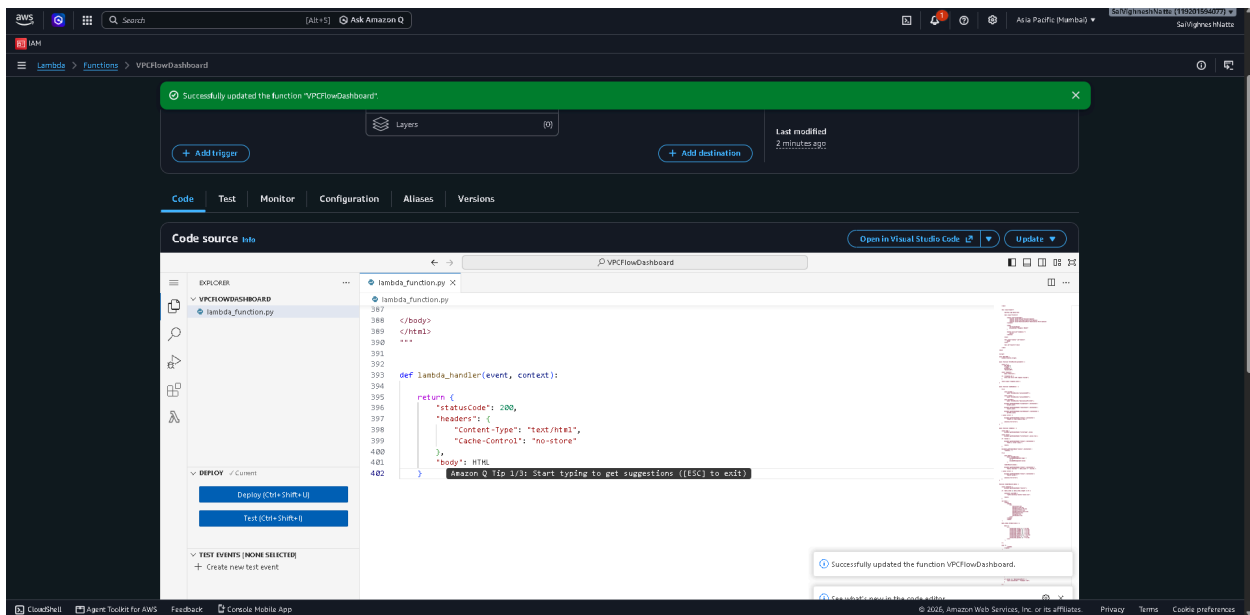
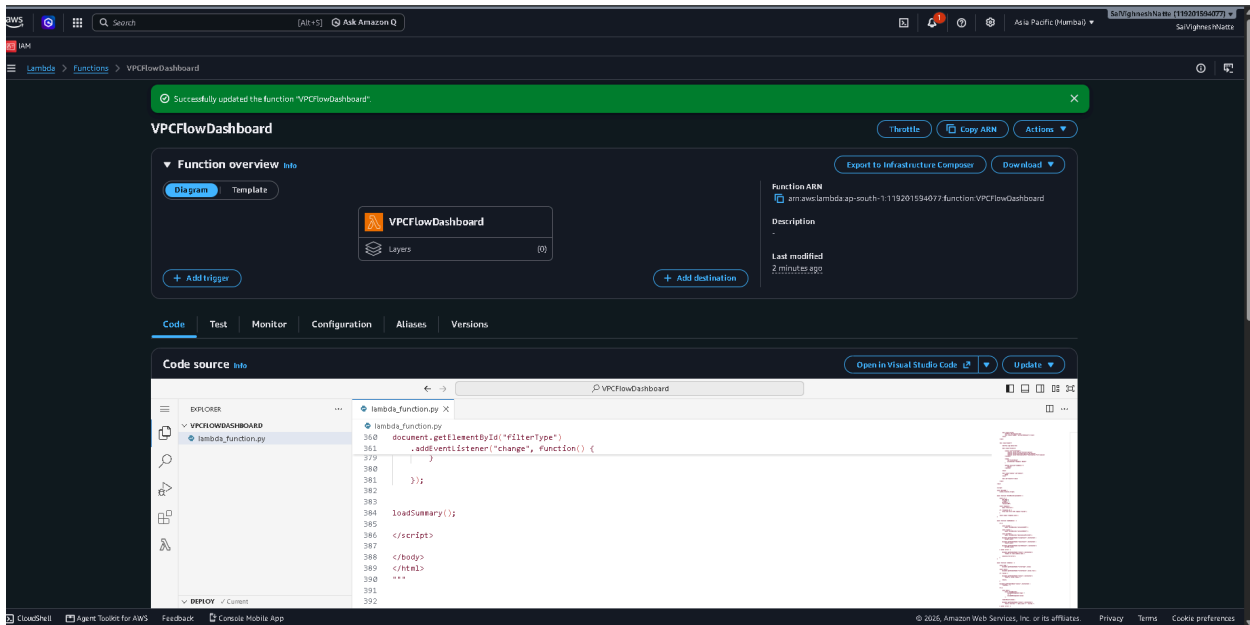
- **Action** → **ACCEPT / REJECT**
- **Source IP**
- **Destination port**

3. Results table

- **Timestamp**
- **Source IP**
- **Destination IP**
- **Source port**
- **Destination port**
- **Protocol**
- **Packets**
- **Bytes**
- **Action**

4. Security alert indicator

- **Highlight REJECT traffic**
- **Show that large S3 objects trigger SNS email**



API Gateway > APIs > Routes - VPCFlowAuditAPI (bu9gll3n)

API Gateway

- APIs
- Custom domain names
- Domain name access associations
- VPC links

API: VPCFlowAuditAPI... (bu9gll3n)

Develop

- Routes
- Authorization
- Integrations
- CORS
- Reimport
- Export

Deploy

- Stages

Monitor

- Metrics
- Logging

Protect

- Throttling

Successfully updated 1 routes

Routes

Routes for VPCFlowAuditAPI

Search

- /dashboard
 - GET
- /logs
 - GET

Create

Route details

GET /dashboard (ID: xl9pauh)

Delete Edit

ARN

arn:aws:apigateway:ap-south-1:api/bu9gll3n/routes/xl9pauh

Authorization

Authorizes protect your API against unauthorized requests. Routes with no authorization attached are open.

No authorizer attached to this route. [Attach authorization](#)

Integration

The integration is the backend resource that this route calls when it receives a request.

bl9laad9 [Configure](#)

CloseShell Agent Toolkit for AWS Feedback Console Mobile App

© 2025 Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

API Gateway > APIs > VPCFlowAuditAPI (bu9gll3n) > Stages

API Gateway

- APIs
- Custom domain names
- Domain name access associations
- VPC links

API: VPCFlowAuditAPI... (bu9gll3n)

Develop

- Routes
- Authorization
- Integrations
- CORS
- Reimport
- Export

Deploy

- Stages

Monitor

- Metrics
- Logging

Protect

- Throttling

Successfully updated 1 routes

Stages

Stages for VPCFlowAuditAPI

Search

- \$default

Create

Stage details

Details

Name	Created	Last updated
\$default	October 4, 2025 3:20 AM	October 4, 2025 3:40 AM

Delete Edit

Invoke URL

<https://bu9gll3n.execute-api-ap-south-1.amazonaws.com>

Description

None

Attached deployment

Automatic Deployment

[Enabled](#)

Deployment ID

Deployment created

bg9Mtu

October 4, 2025 3:40 AM

Deployment description

Automatic deployment triggered by changes to the Api configuration

Stage variables

Find resources

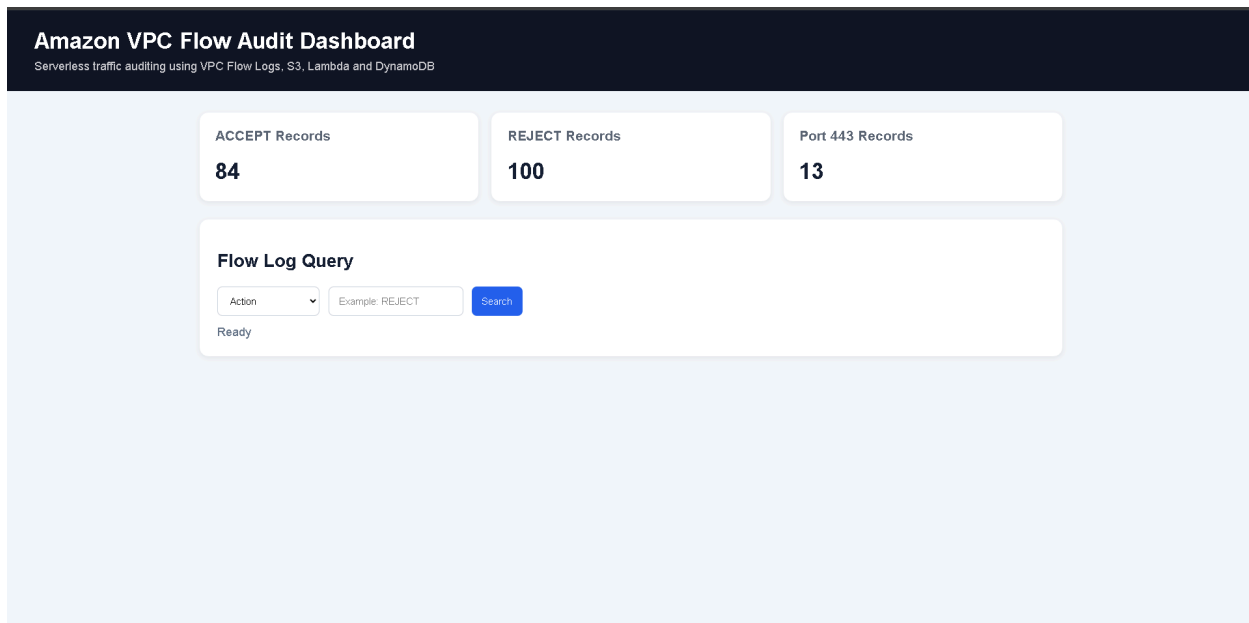
Key	Value
No Stage Variables	

You don't have any stage variables.

Edit

CloseShell Agent Toolkit for AWS Feedback Console Mobile App

© 2025 Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences



Testing:

/dashboard



summary cards load



Action → REJECT



Search



DynamoDB ActionIndex



results displayed

Amazon VPC Flow Audit Dashboard

Serverless traffic auditing using VPC Flow Logs, S3, Lambda and DynamoDB

ACCEPT Records

84

REJECT Records

100

Port 443 Records

13

Flow Log Query

Action

REJECT

Search

Query returned 100 records.

Action	Source IP	Destination IP	Source Port	Destination Port	Protocol	Bytes	Packets
REJECT	162.216.149.16	10.0.1.216	54812	25100	6	44	1
REJECT	212.73.149.9	10.0.1.216	54822	18111	6	52	1
REJECT	31.14.254.26	10.0.1.216	47867	6036	6	40	1
REJECT	35.203.210.245	10.0.1.216	54755	49930	6	44	1
REJECT	147.185.132.219	10.0.1.216	51550	8081	6	44	1
REJECT	147.185.133.224	10.0.1.216	51158	8884	6	44	1
REJECT	162.216.149.71	10.0.1.216	50288	46562	6	44	1
REJECT	173.255.243.63	10.0.1.216	53011	5937	6	44	1
REJECT	212.73.149.3	10.0.1.216	54717	2095	6	52	1
REJECT	212.73.149.35	10.0.1.216	44216	6160	6	52	1
REJECT	212.73.149.37	10.0.1.216	48894	13232	6	52	1
REJECT	35.203.210.209	10.0.1.216	51537	10089	6	44	1
REJECT	147.185.132.190	10.0.1.216	49875	42045	6	44	1
REJECT	147.185.132.199	10.0.1.216	56883	9451	6	44	1

Amazon VPC Flow Audit Dashboard

Serverless traffic auditing using VPC Flow Logs, S3, Lambda and DynamoDB

ACCEPT Records

84

REJECT Records

100

Port 443 Records

13

Flow Log Query

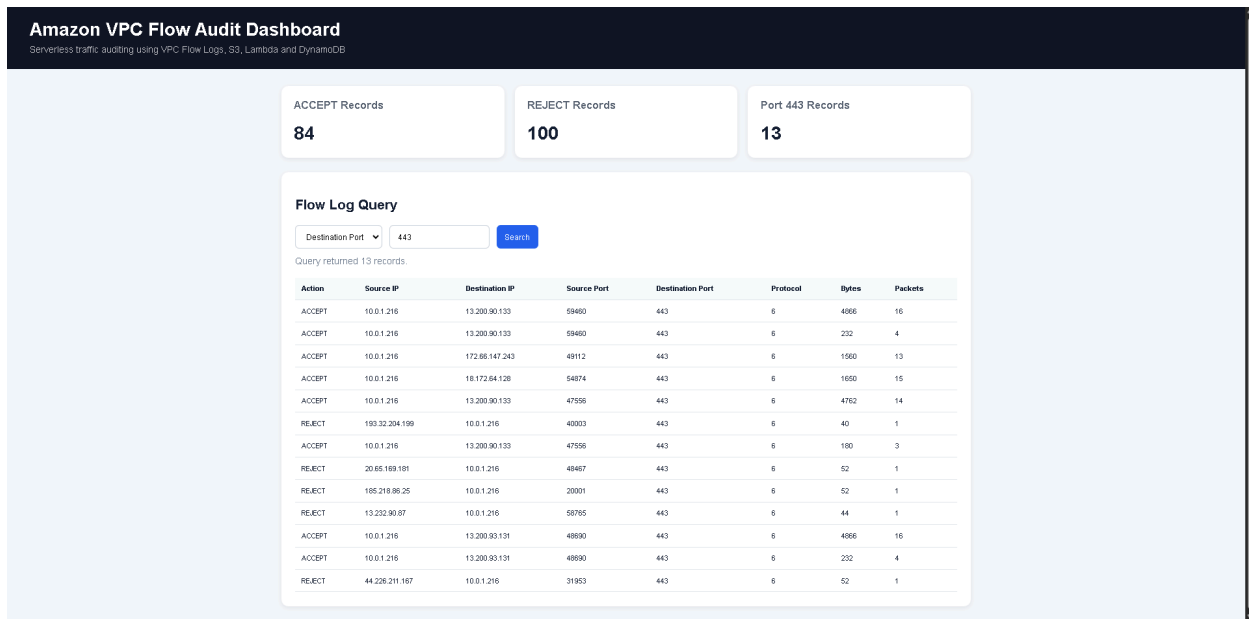
Source IP

162.216.149.16

Search

Query returned 1 records.

Action	Source IP	Destination IP	Source Port	Destination Port	Protocol	Bytes	Packets
REJECT	162.216.149.16	10.0.1.216	54812	25100	6	44	1



PHASE 7 — Security Hardening + Cost Audit

We're going to make the project presentation-ready without adding paid services or unnecessary complexity.

Our goals:

1. Lock down S3
2. Verify DynamoDB security
3. Verify Lambda IAM least privilege
4. Remove unnecessary public CORS
5. Verify API configuration
6. Review cost-sensitive settings
7. Put the project code into GitHub

AWS

Search

[Alt+]/

Asia Pacific (Mumbai)

SaVigraashNatra (11301554777)

Amazon S3

Buckets

vpc-flow-audit-team-382

vpc-flow-audit-team-382

Info

Objects

Metadata

Properties

Permissions

Metrics

Management

File systems

Access Points

Permissions overview

Access finding

Access findings are provided by IAM external access analyzers. Learn more about [how IAM analyzer findings work](#).

View analyzer for app-south-1

Block public access (bucket settings)

Edit

Public access is granted to buckets and objects through access control lists (ACLs), bucket policies, access point policies, or all. In order to ensure that public access to all your S3 buckets and objects is blocked, turn on Block all public access. These settings apply only to this bucket and its access points. AWS recommends that you turn on Block all public access, but before applying any of these settings, ensure that your applications will work correctly without public access. If you require some level of public access to your buckets or objects within, you can customize the individual settings below to suit your specific storage use cases. [Learn more](#).

Block all public access

On

Individual Block Public Access settings for this bucket

Bucket policy

Edit

Delete

The bucket policy, written in JSON, provides access to the objects stored in the bucket. Bucket policies don't apply to objects owned by other accounts. [Learn more](#).

Public access is blocked because Block Public Access settings are turned on for this bucket. To determine which settings are turned on, check your Block Public Access settings for this bucket. Learn more about [using Amazon S3 Block Public Access](#).

```
{
  "Version": "2012-10-17",
  "Id": "AWSLogDeliveryWrite20150313",
  "Statement": [
    {
      "Sid": "AWSLogDeliveryWrite1",
      "Effect": "Allow",
      "Principal": {
        "Service": "delivery.logs.amazonaws.com"
      },
      "Action": "s3:PutObject"
    }
  ]
}
```

Copy

CliShell

Agent Toolkit for AWS

Feedback

Console Mobile App

© 2016, Amazon Web Services, Inc. or its affiliates

Privacy

Terms

Cookie preferences